MODUL 14 · PROYEK BAGIAN 2

Proyek Catatan Kuliah (2): CRUD Supabase & Cache Offline Hive

Melanjutkan proyek Catatan Kuliah: membuat tabel notes dengan RLS dan trigger updated_at, model Note, data source remote (Supabase) dan lokal (Hive), repository dengan fallback offline, NotesController (GetX) dengan pencarian dan optimistic delete, view daftar/form/detail dengan pola MVC, logout yang membersihkan cache, unit test repository dan controller, listing lengkap seluruh file, serta troubleshooting.

Durasi Pertemuan 14 (150 Menit) + Belajar Mandiri

Prasyarat Modul 11: Hive
Modul 12: Supabase
Modul 13: Proyek Catatan Kuliah (1)

Capaian Pembelajaran

- Merancang tabel milik user dengan default auth.uid(), constraint, index, empat policy RLS, dan trigger updated_at.
- Membuat model dengan fromJson, toJson (cache), dan toMap (payload insert/update).
- Memisahkan NoteRemoteDataSource (Supabase) dan NoteLocalDataSource (Hive Map) lalu menggabungkannya di NoteRepository.
- Menerapkan strategi network first, cache fallback beserta penanda mode offline.
- Mengelola state daftar, pencarian, filter, dan optimistic delete dengan rollback di NotesController memakai state reaktif GetX (.obs, RxList).
- Membangun View daftar dengan Obx, RefreshIndicator, empty/loading/error state, Dismissible, serta satu form untuk tambah dan edit dengan navigasi Get.to/Get.back.
- Menulis unit test repository, Hive, dan NotesController memakai fake data source, folder sementara, serta Get.testMode/Get.reset.
- Mendiagnosis error umum seperti 42501 RLS, tabel belum dibuat, dan initialize yang terlewat.

Daftar Isi Modul

1. Tabel notes, RLS & Trigger updated_at
2. Model Note (fromJson, toJson, toMap)
3. NoteRemoteDataSource: CRUD Supabase
4. NoteLocalDataSource: Cache Hive
5. NoteRepository & Mode Offline
6. NotesController & Optimistic Delete

7. HomeView: Daftar, Cari, Filter, Hapus
8. Form Tambah/Edit & Detail Catatan
9. main.dart Final & Logout yang Aman
10. Unit Test Repository, Hive & Controller
11. Listing Lengkap Seluruh File
12. Troubleshooting & Ide Pengembangan
13. Tugas Praktikum
14. Kuis Pemahaman & Kunci Jawaban

1. Tabel notes, RLS & Trigger updated_at

Kita melanjutkan proyek dari Modul 13. Login dan register sudah berjalan, dan sekarang saatnya membuat inti aplikasi: **catatan**. Seperti biasa, kita mulai dari database.

Rencana Modul 14 (urutan kerja)

1. Tabel `notes` + RLS + trigger `updated_at` (section ini)
2. Model `Note`
3. `NoteRemoteDataSource`: CRUD ke Supabase
4. `NoteLocalDataSource`: cache Hive
5. `NoteRepository`: gabungan + fallback offline
6. `NotesController` (GetX): state reaktif, pencarian, optimistic delete
7. `HomeView` final + widget pendukung
8. `NoteFormView` (tambah & edit) + `NoteDetailView`
9. `main.dart` final + logout membersihkan cache
10. Unit test repository, Hive & controller

Desain tabel `notes`

Kolom	Tipe	Keterangan
`id`	uuid	Primary key, dibuat database (`gen_random_uuid()`)
`user_id`	uuid	Pemilik catatan, otomatis `auth.uid()`, FK ke `auth.users`
`title`	text	Judul, wajib, 1–100 karakter
`content`	text	Isi catatan
`category`	text	Salah satu dari Kuliah, Tugas, Ujian, Lainnya
`created_at`	timestamptz	Waktu dibuat, otomatis
`updated_at`	timestamptz	Waktu terakhir diubah, diperbarui oleh trigger

Jalankan SQL berikut di SQL Editor

`supabase/setup.sql`

```
create table if not exists public.notes (
  id uuid primary key default gen_random_uuid(),
```

```
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null check (char_length(title) between 1 and 100),
  content text not null default '',
  category text not null default 'Kuliah'
    check (category in ('Kuliah', 'Tugas', 'Ujian', 'Lainnya')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists notes_user_id_idx on public.notes (user_id);

alter table public.notes enable row level security;

drop policy if exists "User melihat catatannya sendiri" on public.notes;
create policy "User melihat catatannya sendiri"
on public.notes for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "User menambah catatannya sendiri" on public.notes;
create policy "User menambah catatannya sendiri"
on public.notes for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "User mengubah catatannya sendiri" on public.notes;
create policy "User mengubah catatannya sendiri"
on public.notes for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "User menghapus catatannya sendiri" on public.notes;
create policy "User menghapus catatannya sendiri"
on public.notes for delete
to authenticated
using ((select auth.uid()) = user_id);

create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

drop trigger if exists notes_set_updated_at on public.notes;
create trigger notes_set_updated_at
  before update on public.notes
  for each row execute procedure public.set_updated_at();
```

Penjelasan

- **`user_id ... default auth.uid()`** : aplikasi *tidak perlu* mengirim `user_id` saat insert. Database mengisinya dari token login. Ini juga lebih aman karena user tidak bisa "menitipkan" catatan

atas nama orang lain, sebab policy `with check` akan menolaknya.

- `check (category in (...))`: database hanya menerima empat kategori, sama dengan daftar di model Dart.
- `create index ... (user_id)`: setiap query difilter RLS berdasarkan `user_id`. Index membuat filter itu tetap cepat walaupun tabel berisi ribuan catatan.
- **Empat policy** (select, insert, update, delete) semuanya memakai syarat yang sama: `(select auth.uid()) = user_id`.
- **Trigger** `set_updated_at` dijalankan `before update`, sehingga nilai `new.updated_at` diganti `now()` sebelum baris disimpan. Aplikasi tidak perlu (dan tidak bisa salah) mengirim waktu.

Uji RLS dari dua akun

1. Buka **Table Editor → notes** dan pastikan tabel ada dengan label RLS aktif (bukan UNRESTRICTED).
2. Setelah aplikasi selesai, buat dua akun berbeda dan masing-masing menulis catatan.
3. Login bergantian: setiap akun hanya boleh melihat catatannya sendiri. Di Table Editor (mode admin) semua catatan tetap terlihat.

> ⚠️ **Jangan lewati RLS**
>
> Jika lupa `enable row level security`, aplikasi tetap "berjalan normal", tetapi semua user bisa membaca catatan semua orang lewat API. Kesalahan ini tidak terlihat saat diuji dengan satu akun, jadi selalu uji dengan dua akun.

2. Model Note (fromJson, toJson, toMap)

Model `Note` mewakili satu baris tabel `notes`. Model ini dipakai di tiga tempat: data dari Supabase, data dari cache Hive, dan data yang dikirim saat insert/update. Karena itu kita butuh tiga konversi.

File: `lib/models/note.dart`

`lib/models/note.dart`

```dart
/// Model satu catatan kuliah. Satu objek Note = satu baris di tabel `notes`.
class Note {
  const Note({
    required this.id,
    required this.userId,
    required this.title,
    required this.content,
    required this.category,
    required this.createdAt,
    required this.updatedAt,
  });

  /// Pilihan kategori yang tersedia di form.
  static const List<String> categories = [
    'Kuliah',
    'Tugas',
    'Ujian',
    'Lainnya',
  ];

  final String id;
  final String userId;
  final String title;
  final String content;
  final String category;
  final DateTime createdAt;
  final DateTime updatedAt;

  /// Membuat Note dari Map. Dipakai untuk data dari Supabase
  /// (`Map<String, dynamic>`) maupun dari Hive (`Map<dynamic, dynamic>`).
  factory Note.fromJson(Map<dynamic, dynamic> json) {
    return Note(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String? ?? '',
      content: json['content'] as String? ?? '',
      category: json['category'] as String? ?? 'Lainnya',
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }

  /// Seluruh kolom, dipakai untuk menyimpan cache di Hive.
  Map<String, dynamic> toJson() {
    return {
      'id': id,
```

```
      'user_id': userId,
      'title': title,
      'content': content,
      'category': category,
      'created_at': createdAt.toIso8601String(),
      'updated_at': updatedAt.toIso8601String(),
    };
  }

  /// Hanya kolom yang boleh diisi/diubah aplikasi, dipakai untuk
  /// insert/update ke Supabase. id, user_id, created_at, dan updated_at
  /// diisi otomatis oleh database.
  Map<String, dynamic> toMap() {
    return {'title': title, 'content': content, 'category': category};
  }

  Note copyWith({
    String? title,
    String? content,
    String? category,
    DateTime? updatedAt,
  }) {
    return Note(
      id: id,
      userId: userId,
      title: title ?? this.title,
      content: content ?? this.content,
      category: category ?? this.category,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }
}
```

Tiga konversi, tiga tujuan

Method	Arah	Dipakai oleh	Isi
`Note.fromJson(map)`	Map → Note	Remote (Supabase) & Local (Hive)	Semua kolom
`toJson()`	Note → Map	Local (simpan ke Hive)	Semua kolom, tanggal sebagai String ISO
`toMap()`	Note → Map	Remote (update ke Supabase)	Hanya `title`, `content`, `category`

Hal-hal yang perlu diperhatikan

- **Parameter** `Map<dynamic, dynamic>`: Supabase mengembalikan `Map<String, dynamic>`, sedangkan Hive mengembalikan `Map<dynamic, dynamic>` saat membaca Map yang disimpan.

Dengan tipe parameter yang lebih umum, satu factory bisa melayani keduanya tanpa konversi tambahan.

- `json['title'] as String? ?? ''`: aman jika nilai null, tidak langsung crash.
- `DateTime.parse`: Supabase mengirim `timestamptz` sebagai String ISO-8601 (misalnya `2026-10-28T07:15:00.123+00:00`). Kita menyimpannya ke Hive dalam format yang sama, sehingga parsing-nya identik.
- **Kenapa `toMap()` tidak mengirim id, user_id, dan tanggal?** Kolom itu milik database: id dan created_at diisi default, user_id dari `auth.uid()`, dan updated_at dari trigger. Mengirim lebih sedikit data berarti lebih sedikit peluang salah.
- `copyWith` membuat salinan dengan beberapa nilai diganti. Ini dipakai form edit agar objek lama tidak diubah langsung (*immutable*).
- `static const categories` menjadi satu sumber kebenaran untuk dropdown form dan chip filter, dan isinya harus sama dengan `check` di SQL.

Uji cepat di DartPad / main sementara

```
final note = Note.fromJson({
  'id': 'a1',
  'user_id': 'u1',
  'title': 'Basis Data',
  'content': 'Normalisasi 3NF',
  'category': 'Kuliah',
  'created_at': '2026-10-28T07:00:00+00:00',
  'updated_at': '2026-10-28T07:00:00+00:00',
});
debugPrint(note.toMap()); // {title: Basis Data, content: Normalisasi 3NF, category: Kuliah}
debugPrint(Note.fromJson(note.toJson()).title); // Basis Data (bolak-balik aman)
```

3. NoteRemoteDataSource: CRUD Supabase

`NoteRemoteDataSource` berisi semua query Supabase untuk tabel `notes`. Isinya hanyalah penerapan query dasar dari Modul 12 section 7, yang kini dibungkus dalam satu class.

File: `lib/data/note_remote_data_source.dart`

`lib/data/note_remote_data_source.dart`

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';

/// Semua operasi CRUD ke tabel `notes` di Supabase.
/// RLS di database memastikan setiap user hanya melihat barisnya sendiri.
class NoteRemoteDataSource {
  NoteRemoteDataSource(this._client);

  final SupabaseClient _client;

  static const String _table = 'notes';

  Future<List<Note>> fetchNotes() async {
    final rows = await _client
        .from(_table)
        .select()
        .order('updated_at', ascending: false);
    return rows.map(Note.fromJson).toList();
  }

  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final row = await _client
        .from(_table)
        .insert({'title': title, 'content': content, 'category': category})
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<Note> updateNote(Note note) async {
    final row = await _client
        .from(_table)
        .update(note.toMap())
        .eq('id', note.id)
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<void> deleteNote(String id) async {
    await _client.from(_table).delete().eq('id', id);
```

```
  }
}
```

Penjelasan per method

Method	SQL setara	Catatan
`fetchNotes()`	`SELECT * FROM notes ORDER BY updated_at DESC`	Tidak ada `.eq('user_id', ...)` karena RLS sudah memfilter
`createNote()`	`INSERT ... RETURNING *`	`.select().single()` mengembalikan baris baru lengkap dengan id dan tanggal dari database
`updateNote()`	`UPDATE ... WHERE id = ? RETURNING *`	Mengembalikan `updated_at` terbaru dari trigger
`deleteNote()`	`DELETE FROM notes WHERE id = ?`	Filter `id` wajib

> ℹ️ **Kenapa insert tidak mengirim `user_id` ?**
>
> Kolom `user_id` memiliki `default auth.uid()`. Jika Anda tetap ingin mengirimnya secara eksplisit, ambil dari `Supabase.instance.client.auth.currentUser!.id`. Namun mengandalkan default database lebih sederhana dan tidak mungkin salah.

> 💡 **Update jika RLS menolak**
>
> Jika user mencoba mengubah catatan yang bukan miliknya, policy `using` membuat 0 baris cocok. Karena kita memanggil `.single()`, hasil 0 baris berubah menjadi `PostgrestException` (code `PGRST116`), sehingga kegagalan tidak lolos diam-diam.

4. NoteLocalDataSource: Cache Hive

Kita memakai Hive (Modul 11) sebagai **cache**: salinan terakhir catatan dari server yang disimpan di HP. Jika server tidak bisa dihubungi, aplikasi tetap bisa menampilkan catatan dari cache.

Kenapa menyimpan sebagai Map, bukan TypeAdapter?

Di Modul 11 Anda belajar dua cara: menyimpan Map biasa atau membuat `TypeAdapter`. Untuk cache, Map lebih sederhana: model `Note` sudah punya `toJson()` / `fromJson()`, tidak perlu `registerAdapter`, tidak ada `typeId` yang harus dijaga, dan tidak perlu code generation.

```
Box<Map> "notes_cache"
┌──────────────────────────────────────┬──────────────────────────────────────────┐
| key (id catatan)                     | value (Map hasil note.toJson())          |
├──────────────────────────────────────┼──────────────────────────────────────────┤
| "3f6c2a9e-8b1d-4c7a-9e21-..."        | {id: ..., title: "Basis Data", ...}      |
| "a71d0c44-52be-4f0e-8d3a-..."        | {id: ..., title: "Tugas Mobile", ...}    |
└──────────────────────────────────────┴──────────────────────────────────────────┘
```

Memakai **id catatan sebagai key** membuat update dan delete satu catatan menjadi operasi langsung (`put(id)` / `delete(id)`), tanpa perlu mencari indeks.

File: `lib/data/note_local_data_source.dart`

`lib/data/note_local_data_source.dart`

```dart
import 'package:hive_ce/hive_ce.dart';

import '../models/note.dart';

/// Cache catatan di Hive. Setiap catatan disimpan sebagai Map
/// dengan key = id catatan, sehingga tidak perlu TypeAdapter/codegen.
class NoteLocalDataSource {
  NoteLocalDataSource(this._box);

  static const String boxName = 'notes_cache';

  final Box<Map> _box;

  /// Membuka box. Panggil setelah Hive.initFlutter() (atau Hive.init()).
  static Future<NoteLocalDataSource> open() async {
    final box = await Hive.openBox<Map>(boxName);
    return NoteLocalDataSource(box);
  }

  /// Semua catatan di cache, urut dari yang terakhir diubah.
  List<Note> getNotes() {
    final notes = _box.values.map(Note.fromJson).toList();
    notes.sort((a, b) => b.updatedAt.compareTo(a.updatedAt));
    return notes;
  }
```

```dart
  /// Mengganti seluruh isi cache dengan data terbaru dari server.
  Future<void> saveNotes(List<Note> notes) async {
    await _box.clear();
    await _box.putAll({for (final note in notes) note.id: note.toJson()});
  }

  Future<void> saveNote(Note note) => _box.put(note.id, note.toJson());

  Future<void> deleteNote(String id) => _box.delete(id);

  /// Dipanggil saat logout agar data user lain tidak tertinggal.
  Future<void> clear() => _box.clear();
}
```

Penjelasan

- `import 'package:hive_ce/hive_ce.dart'` : file ini cukup memakai Hive inti, tanpa Flutter. Karena itu ia bisa diuji dengan `Hive.init(folderSementara)` di unit test (section 10).
- `static Future<NoteLocalDataSource> open()` : membuka box bersifat async, sedangkan constructor tidak boleh async. Solusinya adalah *factory method* statis yang membuka box lalu membuat objek.
- `getNotes()` bersifat sinkron karena Hive membaca dari memori setelah box dibuka. Hasilnya diurutkan `updatedAt` terbaru, sama dengan urutan dari server.
- `saveNotes()` mengosongkan box lalu `putAll`. Catatan yang sudah dihapus di server (misalnya dari HP lain) ikut hilang dari cache.
- `clear()` dipanggil saat logout. Tanpa ini, user B yang login di HP yang sama bisa sempat melihat catatan user A ketika offline.

Inisialisasi di `main()`

Box harus dibuka sebelum dipakai. Potongan ini akan menjadi bagian dari `main.dart` final di section 9:

```dart
await Hive.initFlutter();                // folder penyimpanan di HP
final localDataSource = await NoteLocalDataSource.open(); // buka box
```

5. NoteRepository & Mode Offline

`NoteRepository` adalah "satu pintu" untuk data catatan. Controller cukup memanggil `repository.getNotes()` tanpa perlu tahu apakah data datang dari Supabase atau dari cache Hive.

Strategi: *network first, cache fallback*

```
getNotes()
   |
   ├─► remote.fetchNotes()  (maks 10 detik)
   |       |
   |       ├─ sukses ──────────► local.saveNotes(data) ──► NotesResult(data, fromCache: false)
   |       |
   |       ├─ PostgrestException ──► rethrow  (server MENJAWAB tapi menolak:
   |       |                                    RLS, tabel tidak ada → harus diperbaiki)
   |       |
   |       └─ error lain ─────► local.getNotes() ──► NotesResult(cache, fromCache: true)
   |          (SocketException,                       → UI menampilkan banner "mode offline"
   |           ClientException,
   |           TimeoutException)
```

File: `lib/data/note_repository.dart`

`lib/data/note_repository.dart`

```dart
import 'dart:async';

import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';
import 'note_local_data_source.dart';
import 'note_remote_data_source.dart';

/// Hasil pengambilan daftar catatan.
class NotesResult {
  const NotesResult(this.notes, {required this.fromCache});

  final List<Note> notes;

  /// true jika server tidak bisa dihubungi dan data diambil dari Hive.
  final bool fromCache;
}

/// Satu pintu untuk semua data catatan. Controller tidak perlu tahu
/// apakah data berasal dari Supabase atau dari cache Hive.
class NoteRepository {
  NoteRepository({
    required this.remote,
    required this.local,
    this.timeout = const Duration(seconds: 10),
  });

  final NoteRemoteDataSource remote;
```

```dart
  final NoteLocalDataSource local;
  final Duration timeout;

  /// Ambil dari server lalu simpan ke cache. Jika gagal karena jaringan,
  /// kembalikan isi cache (mode offline).
  Future<NotesResult> getNotes() async {
    try {
      final notes = await remote.fetchNotes().timeout(timeout);
      await local.saveNotes(notes);
      return NotesResult(notes, fromCache: false);
    } on PostgrestException {
      // Server menjawab tapi menolak (misalnya RLS / tabel tidak ada).
      // Ini bukan masalah jaringan, jadi teruskan errornya.
      rethrow;
    } catch (_) {
      // SocketException, ClientException, TimeoutException, dll.
      return NotesResult(local.getNotes(), fromCache: true);
    }
  }

  Future<Note> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final note = await remote
        .createNote(title: title, content: content, category: category)
        .timeout(timeout);
    await local.saveNote(note);
    return note;
  }

  Future<Note> updateNote(Note note) async {
    final updated = await remote.updateNote(note).timeout(timeout);
    await local.saveNote(updated);
    return updated;
  }

  Future<void> deleteNote(String id) async {
    await remote.deleteNote(id).timeout(timeout);
    await local.deleteNote(id);
  }

  Future<void> clearCache() => local.clear();
}
```

Membedah NoteRepository

- `NotesResult` membungkus dua informasi: daftar catatan *dan* asal datanya. Tanpa flag `fromCache`, UI tidak bisa menampilkan banner offline.
- `.timeout(timeout)`: tanpa batas waktu, request di jaringan yang sangat lambat bisa menggantung lama. Setelah 10 detik muncul `TimeoutException` dan kita beralih ke cache.
- `on PostgrestException { rethrow; }` ditulis *sebelum* `catch (_)`. Error dari database bukan masalah koneksi, sehingga tidak boleh disembunyikan dengan data cache. Kalau disembunyikan, Anda tidak akan sadar policy RLS salah.

- **Operasi tulis (`addNote`, `updateNote`, `deleteNote`)** wajib online. Urutannya: server dulu, cache kemudian. Jika server gagal, exception diteruskan dan cache tidak berubah, sehingga cache tidak pernah berisi data yang tidak ada di server.
- **Field `remote` dan `local` bertipe class konkret**, tetapi tetap bisa diganti di test karena Dart mengizinkan `class Palsu implements NoteRemoteDataSource`.

> ℹ️ **Kenapa tidak memakai package connectivity?**
>
> Status "Wi-Fi tersambung" tidak menjamin internet benar-benar bisa dipakai (misalnya Wi-Fi kampus yang belum login portal). Cara paling jujur adalah langsung mencoba request: jika gagal karena jaringan, kita tahu sedang offline.

> 💡 **Batasan desain ini**
>
> Mode offline di proyek ini bersifat **baca saja**. Menambah atau mengubah catatan saat offline akan gagal dengan pesan yang jelas. Antrean penulisan offline (*offline queue*) dibahas sebagai ide pengembangan di section terakhir.

6. NotesController & Optimistic Delete

`NotesController` adalah **C** (Controller) pada pola MVC untuk fitur catatan. Ia memegang semua state halaman catatan: daftar catatan, status loading, mode offline, pesan error, kata kunci pencarian, dan filter kategori. Controller ini hanya berbicara dengan `NoteRepository` dan tidak mengenal widget sama sekali.

File: `lib/controllers/notes_controller.dart`

`lib/controllers/notes_controller.dart`

```dart
import 'dart:async';

import 'package:get/get.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

class NotesController extends GetxController {
  NotesController(this._repository);

  final NoteRepository _repository;

  // State privat yang reaktif (.obs). Setiap perubahan otomatis
  // memberi tahu Obx yang membacanya.
  final _notes = <Note>[].obs;
  final _isLoading = false.obs;
  final _isOffline = false.obs;
  final _errorMessage = RxnString();
  final _query = ''.obs;
  final _categoryFilter = RxnString(); // null = semua kategori

  bool get isLoading => _isLoading.value;
  bool get isOffline => _isOffline.value;
  String? get errorMessage => _errorMessage.value;
  String get query => _query.value;
  String? get categoryFilter => _categoryFilter.value;
  bool get hasNotes => _notes.isNotEmpty;

  /// Daftar yang ditampilkan di layar setelah difilter pencarian & kategori.
  List<Note> get visibleNotes {
    final q = _query.value.toLowerCase();
    final kategori = _categoryFilter.value;
    return _notes.where((note) {
      final cocokKategori = kategori == null || note.category == kategori;
      final cocokCari =
          q.isEmpty ||
          note.title.toLowerCase().contains(q) ||
          note.content.toLowerCase().contains(q);
      return cocokKategori && cocokCari;
    }).toList();
  }

  Note? findById(String id) {
```

```
    for (final note in _notes) {
      if (note.id == id) return note;
    }
    return null;
  }

  /// Memuat catatan (dipanggil saat HomeView dibuka & saat pull-to-refresh).
  Future<void> loadNotes() async {
    _isLoading.value = true;
    _errorMessage.value = null;

    try {
      final result = await _repository.getNotes();
      _notes.assignAll(result.notes);
      _isOffline.value = result.fromCache;
    } catch (e) {
      _errorMessage.value = _friendlyError(e);
    } finally {
      _isLoading.value = false;
    }
  }

  /// Mengembalikan null jika berhasil, atau pesan error jika gagal.
  Future<String?> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    try {
      final note = await _repository.addNote(
        title: title,
        content: content,
        category: category,
      );
      _notes.insert(0, note);
      _isOffline.value = false;
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  Future<String?> updateNote(Note note) async {
    try {
      final updated = await _repository.updateNote(note);
      _notes.removeWhere((n) => n.id == updated.id);
      _notes.insert(0, updated); // yang baru diubah naik ke atas
      _isOffline.value = false;
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  /// Optimistic delete: hapus dari layar dulu, baru kirim ke server.
  /// Jika server gagal, catatan dikembalikan ke posisi semula (rollback).
  Future<String?> deleteNote(Note note) async {
    final index = _notes.indexWhere((n) => n.id == note.id);
    if (index == -1) return null;
```

```
    _notes.removeAt(index); // UI langsung diperbarui

    try {
      await _repository.deleteNote(note.id);
      return null;
    } catch (e) {
      _notes.insert(index, note); // rollback, UI ikut diperbarui
      return _friendlyError(e);
    }
  }

  void setQuery(String value) => _query.value = value;

  void setCategoryFilter(String? category) => _categoryFilter.value = category;

  /// Dipanggil saat logout: kosongkan state dan cache Hive.
  Future<void> clear() async {
    _notes.clear();
    _query.value = '';
    _categoryFilter.value = null;
    _errorMessage.value = null;
    _isOffline.value = false;
    await _repository.clearCache();
  }

  String _friendlyError(Object e) {
    if (e is PostgrestException) {
      if (e.code == '42501') {
        return 'Akses ditolak oleh Row Level Security. Periksa policy tabel notes.';
      }
      if (e.code == '42P01' || e.code == 'PGRST205') {
        return 'Tabel notes belum dibuat. Jalankan SQL di Supabase.';
      }
      return 'Kesalahan database: ${e.message}';
    }
    if (e is TimeoutException) {
      return 'Server terlalu lama merespons. Coba lagi.';
    }
    return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
  }
}
```

State yang dikelola

Field privat (Rx)	Getter publik	Kegunaan di UI
`_notes` (`RxList<Note>`)	`hasNotes`, `findById`	Semua catatan milik user (sumber data)
(turunan)	`visibleNotes`	`_notes` setelah difilter pencarian + kategori. Inilah yang ditampilkan ListView
`_isLoading` (`RxBool`)	`isLoading`	Spinner saat pertama kali memuat
`_isOffline` (`RxBool`)	`isOffline`	Menampilkan `OfflineBanner`
`_errorMessage` (`RxnString`)	`errorMessage`	Tampilan error + tombol "Coba lagi"
`_query`, `_categoryFilter`	`query`, `categoryFilter`	Isi kolom pencarian dan chip yang terpilih

Polanya sama dengan Modul 08: state disimpan sebagai field **privat** yang reaktif (`.obs`, `RxnString()` untuk nilai yang boleh `null`), lalu dibuka lewat getter **publik** yang hanya-baca. View tidak bisa mengubah state secara langsung. Satu-satunya jalan adalah memanggil method controller seperti `setQuery` atau `deleteNote`.

Karena setiap getter membaca `.value` dari sebuah Rx, `Obx` yang memanggil getter tersebut otomatis "berlangganan" ke Rx itu. Kita tidak perlu memanggil apa pun untuk memberi tahu UI. Mengubah `_isLoading.value` atau memanggil `_notes.insert(...)` sudah cukup.

Pencarian dan filter tanpa query ulang

`visibleNotes` dihitung dari data di memori. Mengetik di kolom pencarian *tidak* mengirim request ke server, sehingga hasilnya instan dan tetap berfungsi saat offline. Untuk ratusan catatan cara ini lebih dari cukup. Untuk data sangat besar, gunakan `.ilike()` di server (Modul 12).

`visibleNotes` adalah getter biasa, bukan Rx. Getter ini membaca tiga Rx sekaligus (`_notes`, `_query`, `_categoryFilter`), jadi `Obx` yang memanggilnya akan rebuild bila salah satu dari ketiganya berubah. Data turunan seperti ini sebaiknya dihitung lewat getter, bukan disimpan sebagai state terpisah yang harus disinkronkan.

Optimistic delete dengan rollback

```
user geser catatan ──► konfirmasi "Hapus"
                          |
                          ▼
   1. simpan index, _notes.removeAt(index)   ← RxList memberi tahu Obx, UI langsung berubah
```

```
                        |
   2. await repository.deleteNote(id)
            |                          |
         sukses                     gagal (offline / RLS)
            |                          |
         selesai          3. _notes.insert(index, note)   ← kembalikan ke posisi semula
                                                          ← Obx rebuild, catatan muncul lagi
                             return pesan error           ← View menampilkan SnackBar
```

Disebut *optimistic* karena kita "optimis" server akan berhasil, sehingga UI diperbarui duluan. Keuntungannya, aplikasi terasa instan. Konsekuensinya, kita **wajib** menyediakan rollback jika ternyata gagal.

Untuk tambah dan edit kita memakai cara sebaliknya (*pessimistic*): tunggu server, baru perbarui list. Alasannya, id dan tanggal catatan baru berasal dari database.

Kontrak nilai kembali: `Future<String?>`

`addNote`, `updateNote`, dan `deleteNote` mengembalikan `null` jika sukses atau `String` pesan error jika gagal. Controller **tidak** menampilkan SnackBar sendiri. View yang memanggilnya cukup menulis:

```dart
final error = await Get.find<NotesController>().addNote(
  title: 'Basis Data',
  content: 'Normalisasi sampai 3NF',
  category: 'Kuliah',
);
if (error == null) {
  // sukses: tampilkan SnackBar lalu Get.back()
} else {
  // tampilkan error di SnackBar
}
```

GetX sebenarnya menyediakan `Get.snackbar` yang bisa dipanggil dari mana saja, termasuk dari controller. Di kursus ini kita sengaja tidak melakukannya. Controller yang tidak menyentuh UI bisa diuji dengan unit test biasa tanpa widget (section 10).

Menerjemahkan error

`_friendlyError` mengubah exception menjadi kalimat untuk user: `42501` (RLS menolak), `42P01` / `PGRST205` (tabel belum dibuat), `TimeoutException`, dan selain itu dianggap masalah koneksi.

Kapan `loadNotes()` dipanggil?

Di Modul 08 kita memuat data awal di `onInit()`. Untuk `NotesController` cara itu tidak cocok. Controller ini didaftarkan sekali di `main()` (section 9), yaitu *sebelum* user login, dan `onInit()` hanya berjalan satu kali seumur aplikasi. Jika memuat di `onInit()`, request dikirim tanpa sesi login dan tidak dikirim ulang setelah user berganti akun. Karena itu `loadNotes()` dipanggil oleh `HomeView` setiap kali halaman itu dibuat, yaitu setiap kali user berhasil login (section 7).

⚠️ **Kesalahan umum**

- **Membuka `_notes` langsung lewat getter publik**: View bisa mengubah list tanpa lewat controller. Di sini View hanya menerima `visibleNotes`, yaitu list baru hasil `where().toList()`.
- **Mengganti list dengan `_notes = [...]`**: field `final` tidak bisa diganti, dan kalaupun bisa, Obx masih mendengarkan RxList yang lama. Gunakan `assignAll`, `insert`, `removeAt`, atau `clear` agar RxList yang sama memberi tahu Obx.
- **Lupa `.value`**: `_isLoading = true` adalah error kompilasi, sedangkan membaca `_query` tanpa `.value` menghasilkan objek `RxString`, bukan teks.
- **Mengimpor `material.dart` atau menyimpan `BuildContext` di controller**: controller jadi sulit diuji. Biarkan View yang menampilkan SnackBar dan bernavigasi.

7. HomeView: Daftar, Cari, Filter, Hapus

Sekarang kita ganti `HomeView` sementara dari Modul 13 dengan halaman daftar catatan lengkap. Inilah **V** (View) pada pola MVC: View hanya menampilkan state dari `NotesController` dan meneruskan aksi user ke controller. Sebelum itu, kita siapkan empat file kecil yang dipakai berulang.

File: `lib/utils/date_format.dart`

Format tanggal sederhana tanpa package tambahan. Jika ingin format yang lebih lengkap, Anda bisa memakai package `intl`.

`lib/utils/date_format.dart`

```dart
/// Format tanggal sederhana tanpa package tambahan, contoh: 5 Okt 2026 14:07.
String formatTanggal(DateTime waktu) {
  const bulan = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'Mei',
    'Jun',
    'Jul',
    'Agu',
    'Sep',
    'Okt',
    'Nov',
    'Des',
  ];
  final lokal = waktu.toLocal();
  final jam = lokal.hour.toString().padLeft(2, '0');
  final menit = lokal.minute.toString().padLeft(2, '0');
  return '${lokal.day} ${bulan[lokal.month - 1]} ${lokal.year} $jam:$menit';
}
```

`toLocal()` penting: Supabase menyimpan waktu dalam UTC, sedangkan pengguna di Purwokerto berada di WIB (UTC+7).

File: `lib/widgets/confirm_dialog.dart`

Dialog "Ya/Batal" dipakai untuk logout dan hapus catatan (dua tempat). Daripada menulis `AlertDialog` berkali-kali, kita buat fungsi bersama:

`lib/widgets/confirm_dialog.dart`

```dart
import 'package:flutter/material.dart';

/// Dialog konfirmasi Ya/Batal yang dipakai ulang di beberapa layar.
/// Mengembalikan true jika pengguna menekan tombol konfirmasi.
Future<bool> showConfirmDialog(
  BuildContext context, {
  required String title,
```

```
  required String message,
  required String confirmLabel,
}) async {
  final hasil = await showDialog<bool>(
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(title),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(dialogContext).pop(false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.of(dialogContext).pop(true),
          child: Text(confirmLabel),
        ),
      ],
    ),
  );
  return hasil ?? false;
}
```

`showDialog<bool>` mengembalikan nilai yang dikirim lewat `Navigator.pop(true/false)`, atau `null` jika user menutup dialog dengan mengetuk area luar. Karena itu ada `?? false`. Di dalam dialog, `Get.back(result: true)` juga bisa dipakai, tetapi `Navigator.of(dialogContext).pop` membuat fungsi ini tetap bisa dipakai tanpa GetX.

File: `lib/widgets/note_card.dart`

`lib/widgets/note_card.dart`

```
import 'package:flutter/material.dart';

import '../models/note.dart';
import '../utils/date_format.dart';

class NoteCard extends StatelessWidget {
  const NoteCard({super.key, required this.note, this.onTap});

  final Note note;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      child: ListTile(
        onTap: onTap,
        contentPadding: const EdgeInsets.fromLTRB(16, 8, 16, 8),
        title: Text(
          note.title,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
```

```
          style: const TextStyle(fontWeight: FontWeight.w600),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 4),
            Text(note.content, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 8),
            Row(
              children: [
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 8,
                    vertical: 2,
                  ),
                  decoration: BoxDecoration(
                    color: scheme.primaryContainer,
                    borderRadius: BorderRadius.circular(12),
                  ),
                  child: Text(
                    note.category,
                    style: TextStyle(
                      fontSize: 12,
                      color: scheme.onPrimaryContainer,
                    ),
                  ),
                ),
                const Spacer(),
                Text(
                  formatTanggal(note.updatedAt),
                  style: Theme.of(context).textTheme.bodySmall,
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/widgets/offline_banner.dart`

`lib/widgets/offline_banner.dart`

```
import 'package:flutter/material.dart';

/// Pita peringatan yang muncul ketika data diambil dari cache Hive.
class OfflineBanner extends StatelessWidget {
  const OfflineBanner({super.key, required this.onRetry});

  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Material(
```

```
      color: scheme.tertiaryContainer,
      child: Padding(
        padding: const EdgeInsets.fromLTRB(16, 4, 8, 4),
        child: Row(
          children: [
            Icon(Icons.cloud_off, color: scheme.onTertiaryContainer),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                'Mode offline: menampilkan data tersimpan.',
                style: TextStyle(color: scheme.onTertiaryContainer),
              ),
            ),
            TextButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}
```

File: `lib/views/home_view.dart` (versi final)

Hapus seluruh isi `HomeView` sementara dari Modul 13 dan ganti dengan kode berikut. Perhatikan bahwa kali ini `HomeView` berupa `StatefulWidget`, karena ia memiliki `TextEditingController` untuk kolom pencarian (state lokal milik halaman ini).

`lib/views/home_view.dart`

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../models/note.dart';
import '../controllers/auth_controller.dart';
import '../controllers/notes_controller.dart';
import '../widgets/confirm_dialog.dart';
import '../widgets/note_card.dart';
import '../widgets/offline_banner.dart';
import 'note_detail_view.dart';
import 'note_form_view.dart';

class HomeView extends StatefulWidget {
  const HomeView({super.key});

  @override
  State<HomeView> createState() => _HomeViewState();
}

class _HomeViewState extends State<HomeView> {
  // Controller global yang didaftarkan di main() dengan Get.put.
  final _auth = Get.find<AuthController>();
  final _notes = Get.find<NotesController>();
  final _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
```

```
    // HomeView dibuat setiap kali user berhasil login, jadi inilah saat
    // yang tepat untuk memuat catatan milik user tersebut.
    _notes.loadNotes();
  }

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  Future<void> _logout() async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Keluar?',
      message: 'Catatan yang tersimpan di perangkat akan dihapus.',
      confirmLabel: 'Keluar',
    );
    if (!yakin) return;

    await _notes.clear(); // kosongkan state + cache Hive
    await _auth.signOut(); // AuthGate otomatis kembali ke LoginView
  }

  Future<bool> _confirmDelete(Note note) {
    return showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
  }

  Future<void> _delete(Note note) async {
    final messenger = ScaffoldMessenger.of(context);
    final error = await _notes.deleteNote(note);
    messenger.showSnackBar(
      SnackBar(
        content: Text(
          error == null
              ? 'Catatan dihapus'
              : 'Gagal menghapus, catatan dikembalikan. $error',
        ),
      ),
    );
  }

  void _openForm() => Get.to(() => const NoteFormView());

  void _openDetail(Note note) => Get.to(() => NoteDetailView(noteId: note.id));

  @override
  Widget build(BuildContext context) {
    final name = _auth.displayName;

    return Scaffold(
      appBar: AppBar(
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
```

```
          children: [
            const Text('Catatan Kuliah'),
            Text('Halo, $name', style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: _logout,
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _openForm,
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
      body: Column(
        children: [
          // Setiap Obx hanya rebuild jika state yang dibacanya berubah.
          Obx(
            () => _notes.isOffline
                ? OfflineBanner(onRetry: _notes.loadNotes)
                : const SizedBox.shrink(),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
            child: Obx(
              () => TextField(
                controller: _searchController,
                onChanged: _notes.setQuery,
                decoration: InputDecoration(
                  hintText: 'Cari judul atau isi catatan...',
                  prefixIcon: const Icon(Icons.search),
                  border: const OutlineInputBorder(),
                  isDense: true,
                  suffixIcon: _notes.query.isEmpty
                      ? null
                      : IconButton(
                          icon: const Icon(Icons.clear),
                          onPressed: () {
                            _searchController.clear();
                            _notes.setQuery('');
                          },
                        ),
                ),
              ),
            ),
          ),
          Obx(
            () => _CategoryFilter(
              selected: _notes.categoryFilter,
              onSelected: _notes.setCategoryFilter,
            ),
          ),
          Expanded(
            child: RefreshIndicator(
              onRefresh: _notes.loadNotes,
```

```
            child: Obx(_buildBody),
          ),
        ),
      ],
    ),
  );
}

Widget _buildBody() {
  // 1. Loading pertama kali (belum ada data sama sekali)
  if (_notes.isLoading && !_notes.hasNotes) {
    return const Center(child: CircularProgressIndicator());
  }

  // 2. Error dan tidak ada data cadangan
  if (_notes.errorMessage != null && !_notes.hasNotes) {
    return _MessageView(
      icon: Icons.error_outline,
      message: _notes.errorMessage!,
      action: FilledButton.icon(
        onPressed: _notes.loadNotes,
        icon: const Icon(Icons.refresh),
        label: const Text('Coba lagi'),
      ),
    );
  }

  final notes = _notes.visibleNotes;

  // 3. Kosong
  if (notes.isEmpty) {
    return _MessageView(
      icon: Icons.note_alt_outlined,
      message: _notes.hasNotes
          ? 'Tidak ada catatan yang cocok.'
          : 'Belum ada catatan.\nTekan tombol + untuk menambah.',
    );
  }

  // 4. Ada data
  return ListView.builder(
    physics: const AlwaysScrollableScrollPhysics(),
    padding: const EdgeInsets.only(top: 4, bottom: 88),
    itemCount: notes.length,
    itemBuilder: (context, index) {
      final note = notes[index];
      return Dismissible(
        key: ValueKey(note.id),
        direction: DismissDirection.endToStart,
        background: Container(
          alignment: Alignment.centerRight,
          padding: const EdgeInsets.only(right: 24),
          color: Theme.of(context).colorScheme.errorContainer,
          child: Icon(
            Icons.delete,
            color: Theme.of(context).colorScheme.onErrorContainer,
          ),
        ),
        confirmDismiss: (_) => _confirmDelete(note),
```

```
          onDismissed: (_) => _delete(note),
          child: NoteCard(note: note, onTap: () => _openDetail(note)),
        );
      },
    );
  }
}

/// Deretan chip untuk memfilter kategori.
class _CategoryFilter extends StatelessWidget {
  const _CategoryFilter({required this.selected, required this.onSelected});

  final String? selected;
  final ValueChanged<String?> onSelected;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 48,
      child: ListView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        children: [
          Padding(
            padding: const EdgeInsets.only(right: 8),
            child: ChoiceChip(
              label: const Text('Semua'),
              selected: selected == null,
              onSelected: (_) => onSelected(null),
            ),
          ),
          for (final category in Note.categories)
            Padding(
              padding: const EdgeInsets.only(right: 8),
              child: ChoiceChip(
                label: Text(category),
                selected: selected == category,
                onSelected: (_) => onSelected(category),
              ),
            ),
        ],
      ),
    );
  }
}

/// Tampilan pesan di tengah layar yang tetap bisa ditarik (pull-to-refresh).
class _MessageView extends StatelessWidget {
  const _MessageView({required this.icon, required this.message, this.action});

  final IconData icon;
  final String message;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    return ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.all(32),
```

```
      children: [
        const SizedBox(height: 48),
        Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
        const SizedBox(height: 16),
        Text(message, textAlign: TextAlign.center),
        if (action != null) ...[
          const SizedBox(height: 16),
          Center(child: action),
        ],
      ],
    );
  }
}
```

Membedah HomeView

Bagian	Penjelasan
`Get.find<AuthController>()`, `Get.find<NotesController>()`	Mengambil controller global yang sudah didaftarkan di `main()` dengan `Get.put` (section 9). Tidak butuh `context`, jadi bisa langsung dijadikan field.
`_notes.loadNotes()` di `initState`	Memuat catatan setiap kali `HomeView` dibuat, yaitu setiap kali user berhasil login. Mengubah Rx di `initState` aman, karena belum ada `Obx` di halaman ini yang sedang dibangun.
`_logout()`	Konfirmasi, lalu `_notes.clear()` (kosongkan state + Hive), lalu `_auth.signOut()`. Tidak ada navigasi manual: `AuthGate` otomatis kembali ke `LoginView`.
Beberapa `Obx` kecil	Banner, kolom pencarian, chip kategori, dan isi daftar masing-masing dibungkus `Obx` sendiri. Setiap `Obx` hanya rebuild jika Rx yang dibacanya berubah. Misalnya `Obx` banner hanya membaca `isOffline`, jadi tidak ikut rebuild saat user mengetik.
`OfflineBanner(onRetry: _notes.loadNotes)`	Banner hanya muncul jika data berasal dari cache. Jika tidak, `Obx` mengembalikan `SizedBox.shrink()` (Obx wajib mengembalikan widget). Tombol "Coba lagi" memanggil ulang server.
`TextField(onChanged: _notes.setQuery)`	Setiap ketikan memperbarui filter. Ikon ✕ muncul jika `query` tidak kosong dan mengosongkan pencarian.
`_CategoryFilter`	Deretan `ChoiceChip`. Memilih "Semua" berarti `null`.
`Obx(_buildBody)`	`Obx` menerima fungsi tanpa parameter yang mengembalikan widget, jadi method `_buildBody` bisa langsung dioper. Semua getter controller yang dibaca di dalamnya ikut didengarkan.
`Get.to(() => const NoteFormView())`	Navigasi GetX tanpa `context`. Halaman detail menerima `noteId` (section 8).
`RefreshIndicator(onRefresh: _notes.loadNotes)`	Tarik ke bawah untuk memuat ulang. Child-nya **harus** bisa di-scroll, karena itu tampilan kosong/error juga berupa `ListView` dengan `AlwaysScrollableScrollPhysics`.

Bagian	Penjelasan
`_buildBody` dengan 4 kondisi	Urutan pengecekan: (1) loading pertama, (2) error tanpa data, (3) kosong, (4) daftar. Jika ada data cache, error tidak menutupi data.
`Dismissible`	`key: ValueKey(note.id)` harus unik dan stabil. `confirmDismiss` menampilkan dialog; jika hasilnya `false`, kartu kembali ke tempatnya. `onDismissed` memanggil optimistic delete.
`ScaffoldMessenger.of(context)` sebelum `await`	Di `_delete`, messenger diambil sebelum menunggu server, sehingga aman dipakai setelahnya tanpa melanggar aturan `context` setelah `await`.
`padding: bottom: 88`	Memberi ruang agar kartu terakhir tidak tertutup FAB.

Hasil di emulator, login sebagai akun demo yang sudah memiliki beberapa catatan:



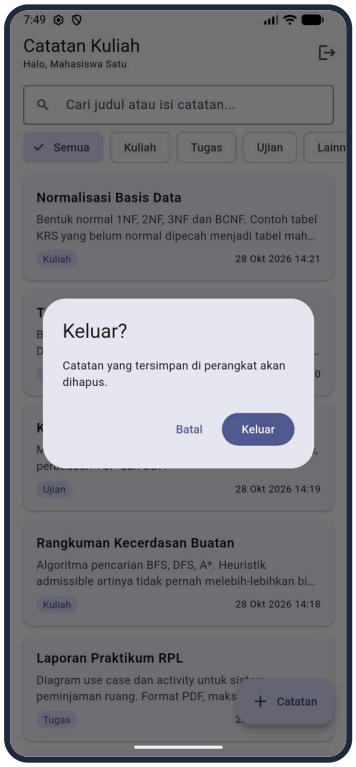
Akun baru: belum ada catatan (empty state)



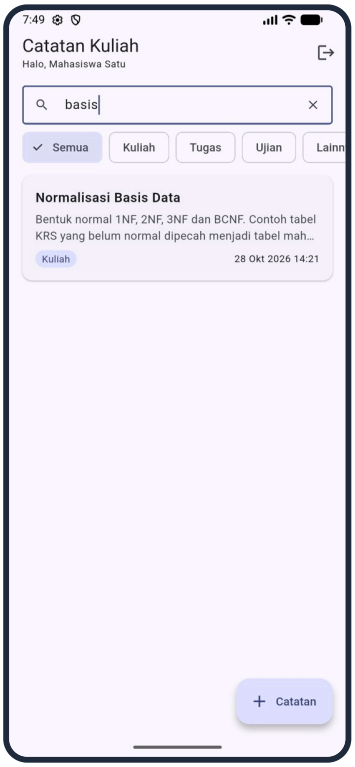
Catatan dari tabel notes, urut terakhir diubah



Chip "Tugas" dipilih



Konfirmasi sebelum logout



Kata kunci "basis" menyaring daftar catatan

⚠️ **Error "A dismissed Dismissible widget is still part of the tree"**

Error ini muncul jika item yang sudah digeser tidak segera dihapus dari list. Optimistic delete di `NotesController` menghapusnya secara sinkron sebelum `await`, sehingga error ini tidak terjadi.

8. Form Tambah/Edit & Detail Catatan

Satu halaman form untuk dua keperluan, tambah dan edit, adalah pola yang sangat umum. Perbedaannya hanya ditentukan oleh ada atau tidaknya parameter `note`. Kedua View di section ini memakai `NotesController` yang sama dengan `HomeView`, diambil dengan `Get.find`.

File: `lib/views/note_form_view.dart`

`lib/views/note_form_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/notes_controller.dart';
import '../models/note.dart';

/// Satu halaman untuk dua keperluan:
/// - note == null  -> mode TAMBAH
/// - note != null  -> mode EDIT
class NoteFormView extends StatefulWidget {
  const NoteFormView({super.key, this.note});

  final Note? note;

  @override
  State<NoteFormView> createState() => _NoteFormViewState();
}

class _NoteFormViewState extends State<NoteFormView> {
  final _notes = Get.find<NotesController>();
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleController;
  late final TextEditingController _contentController;
  late String _category;
  bool _isSaving = false;

  bool get _isEdit => widget.note != null;

  @override
  void initState() {
    super.initState();
    // Mode edit: isi form dengan data lama. Mode tambah: form kosong.
    _titleController = TextEditingController(text: widget.note?.title ?? '');
    _contentController = TextEditingController(
      text: widget.note?.content ?? '',
    );
    _category = widget.note?.category ?? Note.categories.first;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }
```

```dart
  Future<void> _save() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();

    final String? error;
    if (_isEdit) {
      error = await _notes.updateNote(
        widget.note!.copyWith(
          title: title,
          content: content,
          category: _category,
        ),
      );
    } else {
      error = await _notes.addNote(
        title: title,
        content: content,
        category: _category,
      );
    }

    if (!mounted) return;
    setState(() => _isSaving = false);

    final messenger = ScaffoldMessenger.of(context);
    if (error == null) {
      messenger.showSnackBar(
        SnackBar(
          content: Text(_isEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
        ),
      );
      Get.back(); // kembali ke halaman sebelumnya
    } else {
      messenger.showSnackBar(
        SnackBar(
          content: Text(error),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Catatan' : 'Catatan Baru')),
      body: SafeArea(
        child: Form(
          key: _formKey,
          child: ListView(
            padding: const EdgeInsets.all(16),
            children: [
              TextFormField(
                controller: _titleController,
                textInputAction: TextInputAction.next,
```

```
            maxLength: 100,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Judul wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 8),
          DropdownButtonFormField<String>(
            initialValue: _category,
            decoration: const InputDecoration(
              labelText: 'Kategori',
              border: OutlineInputBorder(),
            ),
            items: [
              for (final category in Note.categories)
                DropdownMenuItem(value: category, child: Text(category)),
            ],
            onChanged: (value) {
              if (value != null) setState(() => _category = value);
            },
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _contentController,
            minLines: 6,
            maxLines: 12,
            keyboardType: TextInputType.multiline,
            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              alignLabelWithHint: true,
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Isi catatan wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 24),
          FilledButton.icon(
            onPressed: _isSaving ? null : _save,
            style: FilledButton.styleFrom(
              minimumSize: const Size.fromHeight(48),
            ),
            icon: _isSaving
                ? const SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2.5),
                  )
                : const Icon(Icons.save),
            label: Text(_isSaving ? 'Menyimpan...' : 'Simpan'),
```

```
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Membedah NoteFormView

- `bool get _isEdit => widget.note != null` adalah satu sumber kebenaran untuk judul AppBar, pesan sukses, dan method controller yang dipanggil.
- `TextEditingController` **diisi di** `initState` dengan `widget.note?.title ?? ''`. Mode edit langsung menampilkan data lama, sedangkan mode tambah kosong.
- `DropdownButtonFormField` **dengan** `initialValue` memilih kategori awal. Itemnya diambil dari `Note.categories`.
- `maxLength: 100` menampilkan penghitung karakter dan sejalan dengan `check` di SQL.
- `final _notes = Get.find<NotesController>()` mengambil controller global. Form memanggil `_notes.addNote` atau `_notes.updateNote`, lalu menerima `null` (sukses) atau pesan error.
- `_isSaving` adalah state lokal dengan `setState` (bukan di controller) karena hanya relevan untuk halaman ini: menonaktifkan tombol dan menampilkan spinner. Tidak semua state harus masuk GetX. State yang hanya dipakai satu widget cukup disimpan di `State` widget itu.
- `final String? error;` **diisi di if/else**: Dart mengizinkan variabel `final` diisi tepat sekali di setiap cabang.
- **Setelah** `await` **:** `if (!mounted) return;`, baru `setState`, SnackBar, dan `Get.back()` untuk kembali ke halaman sebelumnya. SnackBar tetap terlihat setelah halaman ditutup karena `ScaffoldMessenger` berada di atas semua halaman.
- **Edit memakai** `copyWith`, sehingga id catatan lama tetap dipakai untuk filter `.eq('id', ...)`.

File: `lib/views/note_detail_view.dart`

`lib/views/note_detail_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/notes_controller.dart';
import '../models/note.dart';
import '../utils/date_format.dart';
import '../widgets/confirm_dialog.dart';
import 'note_form_view.dart';

class NoteDetailView extends GetView<NotesController> {
  const NoteDetailView({super.key, required this.noteId});

  final String noteId;
```

```dart
  Future<void> _delete(BuildContext context, Note note) async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
    if (!yakin || !context.mounted) return;

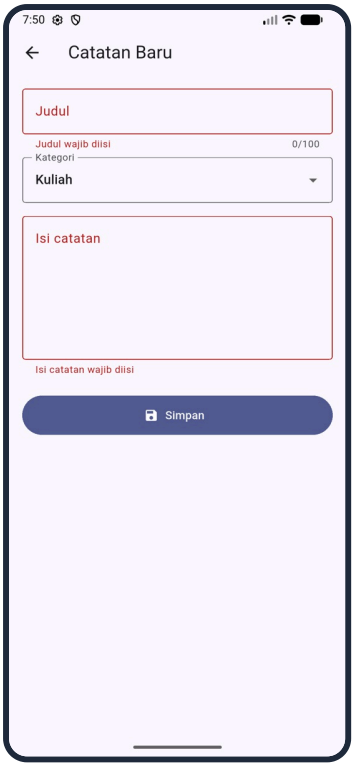
    final messenger = ScaffoldMessenger.of(context);
    // Tutup halaman detail dulu, baru hapus (optimistic delete).
    Get.back();
    final error = await controller.deleteNote(note);
    messenger.showSnackBar(SnackBar(content: Text(error ?? 'Catatan dihapus')));
  }

  @override
  Widget build(BuildContext context) {
    // Obx membaca versi terbaru catatan dari controller, sehingga setelah
    // diedit halaman ini langsung menampilkan perubahan.
    return Obx(() {
      final note = controller.findById(noteId);
      if (note == null) {
        return Scaffold(
          appBar: AppBar(),
          body: const Center(child: Text('Catatan tidak ditemukan.')),
        );
      }
      return _buildDetail(context, note);
    });
  }

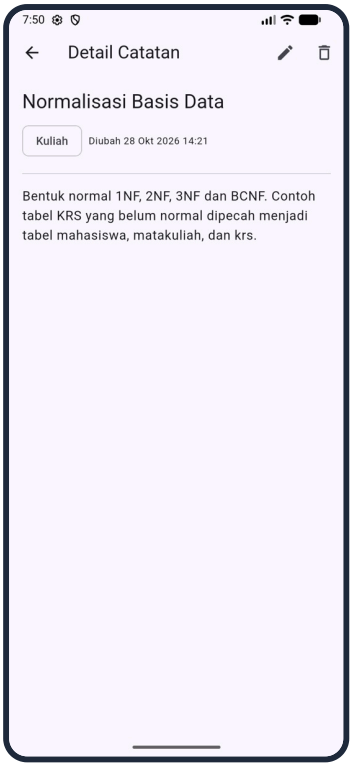
  Widget _buildDetail(BuildContext context, Note note) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Detail Catatan'),
        actions: [
          IconButton(
            tooltip: 'Edit',
            icon: const Icon(Icons.edit),
            onPressed: () => Get.to(() => NoteFormView(note: note)),
          ),
          IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: () => _delete(context, note),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(note.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              Chip(label: Text(note.category)),
```

```
              Text(
                'Diubah ${formatTanggal(note.updatedAt)}',
                style: Theme.of(context).textTheme.bodySmall,
              ),
            ],
          ),
          const Divider(height: 32),
          SelectableText(
            note.content,
            style: Theme.of(context).textTheme.bodyLarge,
          ),
        ],
      ),
    );
  }
}
```

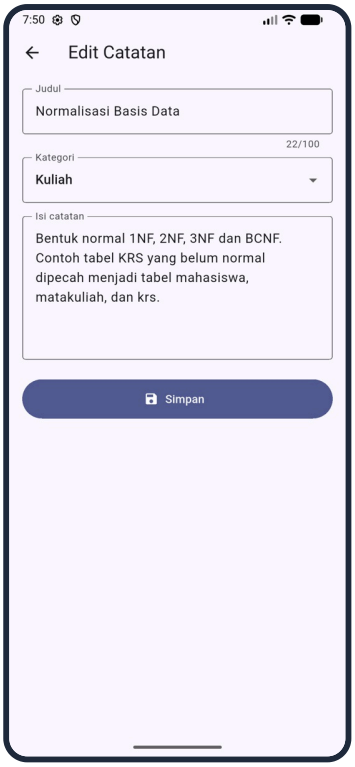
- `extends GetView<NotesController>` : `GetView` adalah `StatelessWidget` yang sudah punya getter `controller` (isinya `Get.find<NotesController>()`). Cocok untuk halaman tanpa state lokal seperti detail ini.
- **Menerima `noteId`, bukan objek `Note`**. Di dalam `Obx`, halaman memanggil `controller.findById(noteId)`, sehingga selalu menampilkan versi *terbaru* dari controller. Setelah diedit dan form ditutup, detail langsung berubah tanpa refresh.
- `Obx` **hanya membaca** `_notes` (lewat `findById`), jadi halaman ini rebuild ketika daftar catatan berubah, tetapi *tidak* ketika `query` atau `categoryFilter` berubah. Inilah alasan kita memakai satu `Obx` kecil per kebutuhan.
- **Hapus dari detail**: halaman ditutup dulu (`Get.back()`), baru optimistic delete berjalan, lalu SnackBar muncul di `HomeView`. Messenger diambil sebelum `Get.back()` karena setelah halaman ditutup `context`-nya tidak boleh dipakai lagi.
- **Edit**: `Get.to(() => NoteFormView(note: note))` membuka form yang sama dalam mode edit.
- `SelectableText` membuat isi catatan bisa disalin, berguna untuk catatan kuliah.



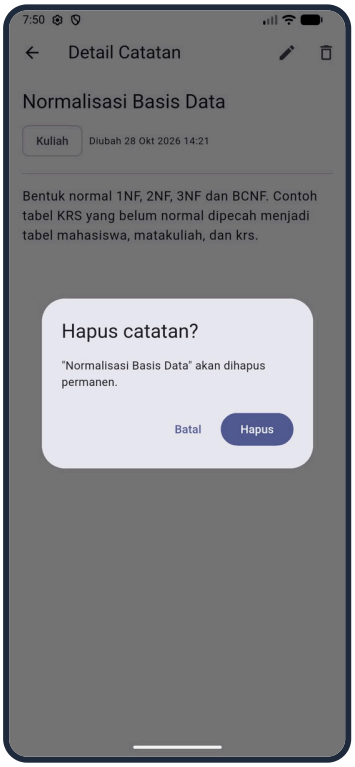
Mode tambah: tombol Simpan ditekan saat form kosong



NoteDetailView dengan tombol Edit dan Hapus



Mode edit: form terisi data lama



Dialog konfirmasi sebelum menghapus



Mode tambah: judul, kategori, dan isi siap disimpan

9. main.dart Final & Logout yang Aman

Semua potongan sudah siap. Langkah terakhir adalah merakitnya di `main.dart`: inisialisasi Hive dan Supabase, membuat objek dari lapisan bawah ke atas, lalu mendaftarkan dua controller global dengan `Get.put`.

File: `lib/main.dart` (versi final)

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'controllers/auth_controller.dart';
import 'controllers/notes_controller.dart';
import 'data/note_local_data_source.dart';
import 'data/note_remote_data_source.dart';
import 'data/note_repository.dart';
import 'services/auth_service.dart';
import 'views/auth_gate.dart';

Future<void> main() async {
  // Wajib sebelum memanggil kode async/plugin sebelum runApp.
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
    return;
  }

  // 1. Hive (cache lokal)
  await Hive.initFlutter();
  final localDataSource = await NoteLocalDataSource.open();

  // 2. Supabase (backend)
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );
  final client = Supabase.instance.client;

  // 3. Rakit lapisan data: Service/DataSource -> Repository
  final authService = AuthService(client);
  final repository = NoteRepository(
    remote: NoteRemoteDataSource(client),
    local: localDataSource,
  );

  // 4. Daftarkan controller global. permanent: true agar tidak pernah
  //    dihapus otomatis oleh GetX selama aplikasi berjalan.
  Get.put(AuthController(authService), permanent: true);
  Get.put(NotesController(repository), permanent: true);
```

```dart
  runApp(const CatatanKuliahApp());
}

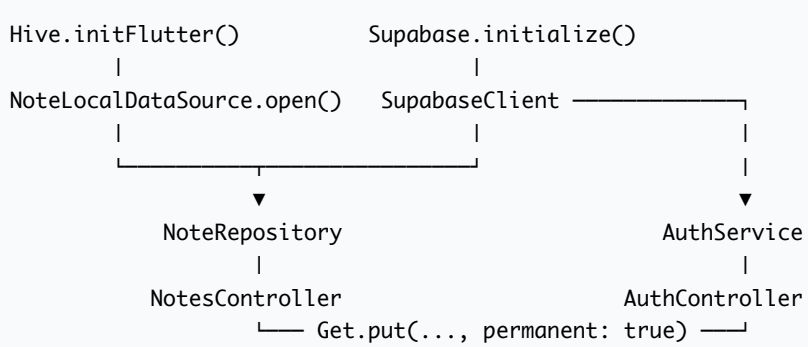
class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

Urutan perakitan (dependency graph)



```
                                  |
            runApp(CatatanKuliahApp) → GetMaterialApp → AuthGate
```

- **Objek dibuat sekali di `main()`** lalu disuntikkan lewat constructor. Tidak ada class yang memanggil `Supabase.instance` atau `Hive.openBox` sendiri.
- **`Get.put(..., permanent: true)`** (Modul 08) mendaftarkan controller ke "gudang" dependency GetX *sebelum* `runApp`. Setelah itu View mana pun bisa mengambilnya dengan `Get.find` atau `GetView`. `permanent: true` mencegah GetX menghapus controller saat halaman ditutup, karena kedua controller ini dipakai selama aplikasi hidup.
- **`GetMaterialApp`** menggantikan `MaterialApp` agar `Get.to`, `Get.back`, dan fitur GetX lain bisa bekerja. Parameternya sama (`title`, `theme`, `home`, ...).
- Dibanding versi Modul 13, yang **bertambah** hanya bagian Hive, repository, dan `NotesController`. Bagian auth tidak berubah sama sekali, dan itulah manfaat arsitektur berlapis.

Alur logout yang aman

```
ikon logout ➔ dialog "Keluar?" ➔ NotesController.clear()
                                     ├─ _notes.clear(), filter direset
                                     └─ repository.clearCache() → Hive box dikosongkan
                                 ➔ AuthController.signOut()
                                     └─ onAuthStateChange: signedOut
                                         └─ _session.value = null
                                             └─ Obx di AuthGate → LoginView
```

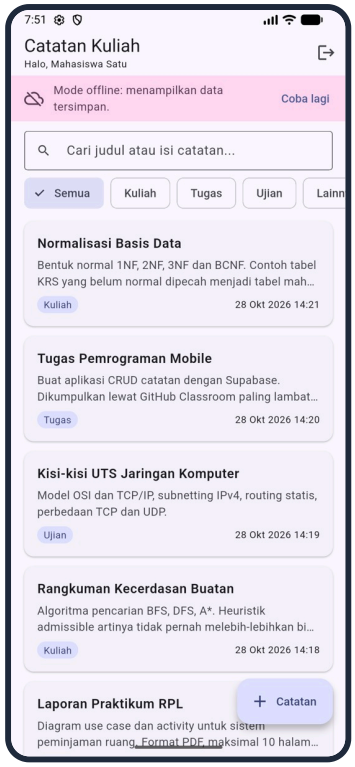
Urutannya penting: cache dibersihkan **sebelum** sesi dihapus. Jika urutannya dibalik, AuthGate langsung membuang `HomeView`, sedangkan `NotesController` (yang `permanent`, jadi tidak ikut dihapus) masih berisi catatan user lama sampai user berikutnya login.

Jalankan aplikasi

```
flutter analyze
flutter run --dart-define-from-file=env.json
```

Uji mode offline

1. Login dan pastikan catatan tampil (data sudah tersimpan di cache).
2. Aktifkan mode pesawat di emulator.
3. Tarik ke bawah (pull-to-refresh). Setelah request gagal, banner **"Mode offline"** muncul dan catatan tetap tampil dari Hive.
4. Coba tambah catatan. Muncul SnackBar error koneksi dan list tidak berubah.
5. Coba hapus catatan (geser). Catatan hilang sebentar lalu muncul kembali (rollback) dengan SnackBar.
6. Matikan mode pesawat dan tekan **Coba lagi** di banner. Banner hilang.



Server tidak bisa dihubungi: catatan tampil dari cache Hive dengan banner offline

10. Unit Test Repository, Hive & Controller

Logika paling penting di proyek ini adalah *fallback* offline di `NoteRepository`, cache Hive, dan state di `NotesController` (pencarian, filter, optimistic delete). Logika seperti ini sulit diuji secara manual (harus menyalakan/mematikan internet berulang kali), tetapi mudah diuji dengan unit test karena repository dan controller tidak bergantung pada widget.

Ide utama: ganti Supabase dengan "server palsu"

Kita membuat `FakeRemoteDataSource` yang `implements NoteRemoteDataSource`. Class ini menyimpan data di List biasa dan bisa diperintah untuk melempar error (misalnya `SocketException`) agar seolah-olah offline. Hive asli tetap dipakai, tetapi diarahkan ke folder sementara dengan `Hive.init(path)`.

Karena fake ini dipakai oleh tiga file test, kita taruh di satu file bantu. File di folder `test/` yang namanya *tidak* berakhiran `_test.dart` tidak dijalankan sebagai test, jadi aman dipakai untuk kode bantu.

File: `test/helpers/fakes.dart`

`test/helpers/fakes.dart`

```dart
import 'package:catatan_kuliah/data/note_remote_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';

/// Membuat Note contoh untuk test. [hari] menentukan tanggal (urutan).
Note buatNote(
  String id, {
  String title = 'Judul',
  String category = 'Kuliah',
  int hari = 1,
}) {
  final waktu = DateTime.utc(2026, 10, hari);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: category,
    createdAt: waktu,
    updatedAt: waktu,
  );
}

/// Pengganti Supabase untuk test. `implements` membuat kita wajib
/// menulis ulang semua method, sehingga tidak ada request jaringan sungguhan.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  List<Note> serverNotes = [];
  Object? errorToThrow; // jika diisi, setiap method melempar error ini

  void _maybeThrow() {
    final error = errorToThrow;
    if (error != null) throw error;
```

```
  }

  @override
  Future<List<Note>> fetchNotes() async {
    _maybeThrow();
    return serverNotes;
  }

  @override
  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    _maybeThrow();
    final note = buatNote('baru', title: title, category: category);
    serverNotes.add(note);
    return note;
  }

  @override
  Future<Note> updateNote(Note note) async {
    _maybeThrow();
    return note;
  }

  @override
  Future<void> deleteNote(String id) async {
    _maybeThrow();
    serverNotes.removeWhere((n) => n.id == id);
  }
}
```

File: `test/note_local_data_source_test.dart`

`test/note_local_data_source_test.dart`

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;

  setUp(() async {
    // Hive.init (bukan initFlutter) karena test berjalan tanpa plugin Flutter.
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
```

```
    await tempDir.delete(recursive: true);
  });

  test('saveNotes lalu getNotes mengembalikan data urut terbaru', () async {
    await local.saveNotes([
      buatNote('a', title: 'Lama', hari: 1),
      buatNote('b', title: 'Baru', hari: 5),
    ]);

    final notes = local.getNotes();

    expect(notes.map((n) => n.id), ['b', 'a']);
    expect(notes.first.title, 'Baru');
  });

  test('saveNotes mengganti seluruh isi cache', () async {
    await local.saveNotes([buatNote('a'), buatNote('b')]);
    await local.saveNotes([buatNote('c')]);

    expect(local.getNotes().map((n) => n.id), ['c']);
  });

  test('saveNote, deleteNote, dan clear', () async {
    await local.saveNote(buatNote('a'));
    await local.saveNote(buatNote('a', title: 'Diubah'));
    expect(local.getNotes().single.title, 'Diubah');

    await local.saveNote(buatNote('b'));
    await local.deleteNote('a');
    expect(local.getNotes().map((n) => n.id), ['b']);

    await local.clear();
    expect(local.getNotes(), isEmpty);
  });
}
```

File: `test/note_repository_test.dart`

`test/note_repository_test.dart`

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;
  late FakeRemoteDataSource remote;
  late NoteRepository repository;

  setUp(() async {
    tempDir = await Directory.systemTemp.createTemp('repo_test_');
```

```
  Hive.init(tempDir.path);
  local = await NoteLocalDataSource.open();
  remote = FakeRemoteDataSource();
  repository = NoteRepository(remote: remote, local: local);
});

tearDown(() async {
  await Hive.deleteFromDisk();
  await tempDir.delete(recursive: true);
});

test('online: data server dikembalikan dan disimpan ke cache', () async {
  remote.serverNotes = [buatNote('1'), buatNote('2')];

  final result = await repository.getNotes();

  expect(result.fromCache, isFalse);
  expect(result.notes, hasLength(2));
  expect(local.getNotes(), hasLength(2));
});

test('offline: jika jaringan gagal, data diambil dari cache', () async {
  await local.saveNotes([buatNote('cache-1', title: 'Dari Hive')]);
  remote.errorToThrow = const SocketException('Tidak ada internet');

  final result = await repository.getNotes();

  expect(result.fromCache, isTrue);
  expect(result.notes.single.title, 'Dari Hive');
});

test('error dari server (PostgrestException) tidak disembunyikan', () async {
  remote.errorToThrow = const PostgrestException(
    message: 'permission denied',
    code: '42501',
  );

  expect(repository.getNotes(), throwsA(isA<PostgrestException>()));
});

test('addNote dan deleteNote ikut memperbarui cache', () async {
  final note = await repository.addNote(
    title: 'Basis Data',
    content: 'Normalisasi',
    category: 'Kuliah',
  );
  expect(local.getNotes().single.id, note.id);

  await repository.deleteNote(note.id);
  expect(local.getNotes(), isEmpty);
});

test('deleteNote gagal: cache tidak berubah', () async {
  await local.saveNotes([buatNote('1')]);
  remote.errorToThrow = const SocketException('offline');

  await expectLater(
    repository.deleteNote('1'),
    throwsA(isA<SocketException>()),
```

```
    );
    expect(local.getNotes(), hasLength(1));
  });
}
```

File: `test/notes_controller_test.dart`

Inilah keuntungan controller yang tidak menyentuh UI: `NotesController` bisa diuji seperti class Dart biasa. Kita hanya perlu mengaktifkan mode test GetX dan membersihkan semua instance setelah setiap test.

`test/notes_controller_test.dart`

```dart
import 'dart:io';

import 'package:catatan_kuliah/controllers/notes_controller.dart';
import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:get/get.dart';
import 'package:hive_ce/hive_ce.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late FakeRemoteDataSource remote;
  late NotesController controller;

  setUp(() async {
    Get.testMode = true;
    tempDir = await Directory.systemTemp.createTemp('controller_test_');
    Hive.init(tempDir.path);
    remote = FakeRemoteDataSource()
      ..serverNotes = [
        buatNote('1', title: 'Basis Data', hari: 3),
        buatNote('2', title: 'Tugas Mobile', category: 'Tugas', hari: 2),
        buatNote('3', title: 'Kisi-kisi UTS', category: 'Ujian', hari: 1),
      ];
    final repository = NoteRepository(
      remote: remote,
      local: await NoteLocalDataSource.open(),
    );
    controller = Get.put(NotesController(repository));
  });

  tearDown(() async {
    Get.reset(); // hapus semua controller agar setiap test terisolasi
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('loadNotes mengisi daftar dan mematikan loading', () async {
    final future = controller.loadNotes();
    expect(controller.isLoading, isTrue);

    await future;
```

```
    expect(controller.isLoading, isFalse);
    expect(controller.isOffline, isFalse);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('loadNotes offline: data cache dipakai dan isOffline true', () async {
    await controller.loadNotes(); // online dulu, cache terisi
    remote.errorToThrow = const SocketException('offline');

    await controller.loadNotes();

    expect(controller.isOffline, isTrue);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('setQuery dan setCategoryFilter menyaring visibleNotes', () async {
    await controller.loadNotes();

    controller.setQuery('mobile');
    expect(controller.visibleNotes.single.id, '2');

    controller.setQuery('');
    controller.setCategoryFilter('Ujian');
    expect(controller.visibleNotes.single.id, '3');

    controller.setCategoryFilter(null);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('deleteNote gagal: catatan dikembalikan ke posisi semula', () async {
    await controller.loadNotes();
    final note = controller.findById('2')!;
    remote.errorToThrow = const SocketException('offline');

    final future = controller.deleteNote(note);
    // Optimistic: sebelum server menjawab, catatan sudah hilang.
    expect(controller.findById('2'), isNull);

    final error = await future;

    expect(error, isNotNull);
    expect(controller.visibleNotes.map((n) => n.id), ['1', '2', '3']);
  });
}
```

Penjelasan

- `setUp` / `tearDown` dijalankan sebelum/sesudah *setiap* test: buat folder sementara, `Hive.init`, buka box, lalu hapus semuanya. Setiap test mulai dari kondisi bersih.
- `Hive.init(tempDir.path)` dipakai, bukan `Hive.initFlutter()`, karena di unit test tidak ada plugin `path_provider`.
- **Test "offline"** adalah inti modul ini: cache diisi, server dibuat gagal dengan `SocketException`, lalu kita pastikan `fromCache` bernilai `true` dan data cache yang dikembalikan.
- **Test PostgrestException** memastikan error RLS *tidak* disembunyikan oleh cache.

- `expectLater(future, throwsA(...))` menunggu Future selesai lalu memeriksa exception-nya.
- `Get.testMode = true` memberi tahu GetX bahwa kode berjalan di lingkungan test (tanpa `GetMaterialApp`), sehingga fitur yang butuh navigator tidak membuat test gagal.
- `Get.put(NotesController(repository))` mendaftarkan controller seperti di `main()`, tetapi dengan repository yang memakai fake data source. `Get.reset()` di `tearDown` menghapus semua instance agar setiap test mulai dari controller baru.
- **Membaca getter langsung**: `controller.isLoading`, `controller.visibleNotes`, dan seterusnya bisa dibaca tanpa `Obx`. Test `loadNotes` bahkan memeriksa `isLoading` bernilai `true` sebelum `await`, lalu `false` sesudahnya.
- **Test optimistic delete** memastikan dua hal: catatan langsung hilang sebelum server menjawab, lalu kembali ke posisi semula (urutan `['1', '2', '3']`) ketika server gagal.

Jalankan

```
flutter test
```

Hasil yang diharapkan:

```
00:01 +12: All tests passed!
```

> ℹ️ **Kenapa tidak ada widget test bawaan?**
>
> `flutter create` membuat `test/widget_test.dart` untuk aplikasi counter. File itu tidak relevan lagi dan akan gagal, jadi hapus saja. Widget test untuk halaman Login/Home dibahas di modul Testing.

11. Listing Lengkap Seluruh File

Section ini berisi **seluruh** file aplikasi Catatan Kuliah dalam bentuk final dan lengkap, siap disalin. Gunakan untuk membandingkan jika aplikasi Anda error, atau untuk memulai ulang dari nol. Semua kode di bawah sudah diverifikasi dengan `flutter analyze` (tanpa masalah) dan `flutter test` (12 test lulus).

Langkah dari proyek kosong

```
flutter create catatan_kuliah
cd catatan_kuliah
flutter pub add supabase_flutter get hive_ce hive_ce_flutter
rm test/widget_test.dart
mkdir -p lib/config lib/models lib/services lib/data lib/controllers lib/views lib/widgets lib/utils
test/helpers supabase
```

Lalu buat setiap file di bawah ini, jalankan SQL di Supabase, isi `env.json`, dan jalankan `flutter run --dart-define-from-file=env.json`.

File non-Dart

`env.example.json`

```
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Salin menjadi `env.json` lalu isi dengan nilai project Anda. Tambahkan ke `.gitignore`:

`.gitignore`

```
# Kunci Supabase
env.json
```

Di `android/app/src/main/AndroidManifest.xml`, tepat di bawah tag pembuka `<manifest ...>`:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

SQL lengkap (jalankan sekali di SQL Editor; bagian `mata_kuliah` adalah tabel demo Modul 12 dan boleh dilewati):

`supabase/setup.sql`

```
-- ============================================================================
-- Setup Supabase untuk Modul 12 (demo) dan Modul 13-14 (Catatan Kuliah)
-- Jalankan seluruh isi file ini sekali di Dashboard > SQL Editor > Run.
-- Aman dijalankan ulang (idempotent).
-- ============================================================================
```

```sql
-- ---------------------------------------------------------------------
-- MODUL 12: tabel demo mata_kuliah
-- ---------------------------------------------------------------------
create table if not exists public.mata_kuliah (
  id bigint generated always as identity primary key,
  kode text not null unique,
  nama text not null,
  sks int not null default 3 check (sks between 1 and 6),
  semester int not null check (semester between 1 and 8),
  created_at timestamptz not null default now()
);

alter table public.mata_kuliah enable row level security;

drop policy if exists "Semua orang boleh membaca mata kuliah" on public.mata_kuliah;
create policy "Semua orang boleh membaca mata kuliah"
on public.mata_kuliah for select
to anon, authenticated
using (true);

drop policy if exists "User login boleh menambah mata kuliah" on public.mata_kuliah;
create policy "User login boleh menambah mata kuliah"
on public.mata_kuliah for insert
to authenticated
with check (true);

insert into public.mata_kuliah (kode, nama, sks, semester) values
  ('INF101', 'Algoritma dan Pemrograman', 4, 1),
  ('INF102', 'Matematika Diskrit', 3, 1),
  ('INF201', 'Struktur Data', 3, 2),
  ('INF202', 'Basis Data', 3, 3),
  ('INF301', 'Pemrograman Mobile', 3, 5),
  ('INF302', 'Jaringan Komputer', 3, 4),
  ('INF303', 'Kecerdasan Buatan', 3, 5),
  ('INF304', 'Rekayasa Perangkat Lunak', 3, 4)
on conflict (kode) do nothing;

-- Aktifkan Realtime untuk tabel mata_kuliah (dipakai contoh .stream()).
do $$
begin
  if not exists (
    select 1 from pg_publication_tables
    where pubname = 'supabase_realtime'
      and schemaname = 'public' and tablename = 'mata_kuliah'
  ) then
    alter publication supabase_realtime add table public.mata_kuliah;
  end if;
end;
$$;

-- ---------------------------------------------------------------------
-- MODUL 13: tabel profiles + trigger handle_new_user
-- ---------------------------------------------------------------------
create table if not exists public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  full_name text,
  created_at timestamptz not null default now()
);
```

```sql
alter table public.profiles enable row level security;

drop policy if exists "User melihat profilnya sendiri" on public.profiles;
create policy "User melihat profilnya sendiri"
on public.profiles for select
to authenticated
using ((select auth.uid()) = id);

drop policy if exists "User mengubah profilnya sendiri" on public.profiles;
create policy "User mengubah profilnya sendiri"
on public.profiles for update
to authenticated
using ((select auth.uid()) = id)
with check ((select auth.uid()) = id);

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, new.raw_user_meta_data ->> 'full_name');
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

-- ------------------------------------------------------------------------
-- MODUL 14: tabel notes + RLS + trigger updated_at
-- ------------------------------------------------------------------------
create table if not exists public.notes (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
  title text not null check (char_length(title) between 1 and 100),
  content text not null default '',
  category text not null default 'Kuliah'
    check (category in ('Kuliah', 'Tugas', 'Ujian', 'Lainnya')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists notes_user_id_idx on public.notes (user_id);

alter table public.notes enable row level security;

drop policy if exists "User melihat catatannya sendiri" on public.notes;
create policy "User melihat catatannya sendiri"
on public.notes for select
to authenticated
using ((select auth.uid()) = user_id);

drop policy if exists "User menambah catatannya sendiri" on public.notes;
```

```sql
create policy "User menambah catatannya sendiri"
on public.notes for insert
to authenticated
with check ((select auth.uid()) = user_id);

drop policy if exists "User mengubah catatannya sendiri" on public.notes;
create policy "User mengubah catatannya sendiri"
on public.notes for update
to authenticated
using ((select auth.uid()) = user_id)
with check ((select auth.uid()) = user_id);

drop policy if exists "User menghapus catatannya sendiri" on public.notes;
create policy "User menghapus catatannya sendiri"
on public.notes for delete
to authenticated
using ((select auth.uid()) = user_id);

create or replace function public.set_updated_at()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

drop trigger if exists notes_set_updated_at on public.notes;
create trigger notes_set_updated_at
  before update on public.notes
  for each row execute procedure public.set_updated_at();
```

Konfigurasi & entry point

lib/main.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'config/env.dart';
import 'controllers/auth_controller.dart';
import 'controllers/notes_controller.dart';
import 'data/note_local_data_source.dart';
import 'data/note_remote_data_source.dart';
import 'data/note_repository.dart';
import 'services/auth_service.dart';
import 'views/auth_gate.dart';

Future<void> main() async {
  // Wajib sebelum memanggil kode async/plugin sebelum runApp.
  WidgetsFlutterBinding.ensureInitialized();

  if (!Env.isConfigured) {
    runApp(const ConfigErrorApp());
```

```
    return;
  }

  // 1. Hive (cache lokal)
  await Hive.initFlutter();
  final localDataSource = await NoteLocalDataSource.open();

  // 2. Supabase (backend)
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );
  final client = Supabase.instance.client;

  // 3. Rakit lapisan data: Service/DataSource -> Repository
  final authService = AuthService(client);
  final repository = NoteRepository(
    remote: NoteRemoteDataSource(client),
    local: localDataSource,
  );

  // 4. Daftarkan controller global. permanent: true agar tidak pernah
  //    dihapus otomatis oleh GetX selama aplikasi berjalan.
  Get.put(AuthController(authService), permanent: true);
  Get.put(NotesController(repository), permanent: true);

  runApp(const CatatanKuliahApp());
}

class CatatanKuliahApp extends StatelessWidget {
  const CatatanKuliahApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'Catatan Kuliah',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const AuthGate(),
    );
  }
}

/// Ditampilkan jika aplikasi dijalankan tanpa --dart-define-from-file.
class ConfigErrorApp extends StatelessWidget {
  const ConfigErrorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi.\n\n'
```

```
              'Jalankan dengan:\n'
              'flutter run --dart-define-from-file=env.json',
              textAlign: TextAlign.center,
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/config/env.dart

```dart
/// Konfigurasi Supabase yang dibaca saat build lewat --dart-define.
///
/// Jalankan aplikasi dengan:
///   flutter run --dart-define-from-file=env.json
class Env {
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  /// true jika kedua nilai sudah diisi.
  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

Model

lib/models/note.dart

```dart
/// Model satu catatan kuliah. Satu objek Note = satu baris di tabel `notes`.
class Note {
  const Note({
    required this.id,
    required this.userId,
    required this.title,
    required this.content,
    required this.category,
    required this.createdAt,
    required this.updatedAt,
  });

  /// Pilihan kategori yang tersedia di form.
  static const List<String> categories = [
    'Kuliah',
    'Tugas',
    'Ujian',
    'Lainnya',
  ];

  final String id;
  final String userId;
  final String title;
  final String content;
  final String category;
```

```dart
  final DateTime createdAt;
  final DateTime updatedAt;

  /// Membuat Note dari Map. Dipakai untuk data dari Supabase
  /// (`Map<String, dynamic>`) maupun dari Hive (`Map<dynamic, dynamic>`).
  factory Note.fromJson(Map<dynamic, dynamic> json) {
    return Note(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String? ?? '',
      content: json['content'] as String? ?? '',
      category: json['category'] as String? ?? 'Lainnya',
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }

  /// Seluruh kolom, dipakai untuk menyimpan cache di Hive.
  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'user_id': userId,
      'title': title,
      'content': content,
      'category': category,
      'created_at': createdAt.toIso8601String(),
      'updated_at': updatedAt.toIso8601String(),
    };
  }

  /// Hanya kolom yang boleh diisi/diubah aplikasi, dipakai untuk
  /// insert/update ke Supabase. id, user_id, created_at, dan updated_at
  /// diisi otomatis oleh database.
  Map<String, dynamic> toMap() {
    return {'title': title, 'content': content, 'category': category};
  }

  Note copyWith({
    String? title,
    String? content,
    String? category,
    DateTime? updatedAt,
  }) {
    return Note(
      id: id,
      userId: userId,
      title: title ?? this.title,
      content: content ?? this.content,
      category: category ?? this.category,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }
}
```

Service & data

lib/services/auth_service.dart

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

/// Lapisan paling bawah untuk urusan autentikasi.
/// Hanya class ini yang berbicara langsung dengan Supabase Auth.
class AuthService {
  AuthService(this._client);

  final SupabaseClient _client;

  GoTrueClient get _auth => _client.auth;

  /// Stream yang memancarkan event setiap status login berubah
  /// (initialSession, signedIn, signedOut, tokenRefreshed, ...).
  Stream<AuthState> get authStateChanges => _auth.onAuthStateChange;

  Session? get currentSession => _auth.currentSession;

  User? get currentUser => _auth.currentUser;

  /// Mendaftarkan akun baru. Nama lengkap dikirim sebagai metadata
  /// (raw_user_meta_data) dan nanti disalin trigger ke tabel profiles.
  Future<AuthResponse> signUp({
    required String fullName,
    required String email,
    required String password,
  }) {
    return _auth.signUp(
      email: email,
      password: password,
      data: {'full_name': fullName},
    );
  }

  Future<AuthResponse> signIn({
    required String email,
    required String password,
  }) {
    return _auth.signInWithPassword(email: email, password: password);
  }

  Future<void> signOut() => _auth.signOut();
}
```

lib/data/note_remote_data_source.dart

```dart
import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';

/// Semua operasi CRUD ke tabel `notes` di Supabase.
/// RLS di database memastikan setiap user hanya melihat barisnya sendiri.
class NoteRemoteDataSource {
  NoteRemoteDataSource(this._client);
```

```dart
  final SupabaseClient _client;

  static const String _table = 'notes';

  Future<List<Note>> fetchNotes() async {
    final rows = await _client
        .from(_table)
        .select()
        .order('updated_at', ascending: false);
    return rows.map(Note.fromJson).toList();
  }

  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final row = await _client
        .from(_table)
        .insert({'title': title, 'content': content, 'category': category})
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<Note> updateNote(Note note) async {
    final row = await _client
        .from(_table)
        .update(note.toMap())
        .eq('id', note.id)
        .select()
        .single();
    return Note.fromJson(row);
  }

  Future<void> deleteNote(String id) async {
    await _client.from(_table).delete().eq('id', id);
  }
}
```

lib/data/note_local_data_source.dart

```dart
import 'package:hive_ce/hive_ce.dart';

import '../models/note.dart';

/// Cache catatan di Hive. Setiap catatan disimpan sebagai Map
/// dengan key = id catatan, sehingga tidak perlu TypeAdapter/codegen.
class NoteLocalDataSource {
  NoteLocalDataSource(this._box);

  static const String boxName = 'notes_cache';

  final Box<Map> _box;

  /// Membuka box. Panggil setelah Hive.initFlutter() (atau Hive.init()).
  static Future<NoteLocalDataSource> open() async {
```

```
    final box = await Hive.openBox<Map>(boxName);
    return NoteLocalDataSource(box);
  }

  /// Semua catatan di cache, urut dari yang terakhir diubah.
  List<Note> getNotes() {
    final notes = _box.values.map(Note.fromJson).toList();
    notes.sort((a, b) => b.updatedAt.compareTo(a.updatedAt));
    return notes;
  }

  /// Mengganti seluruh isi cache dengan data terbaru dari server.
  Future<void> saveNotes(List<Note> notes) async {
    await _box.clear();
    await _box.putAll({for (final note in notes) note.id: note.toJson()});
  }

  Future<void> saveNote(Note note) => _box.put(note.id, note.toJson());

  Future<void> deleteNote(String id) => _box.delete(id);

  /// Dipanggil saat logout agar data user lain tidak tertinggal.
  Future<void> clear() => _box.clear();
}
```

lib/data/note_repository.dart

```
import 'dart:async';

import 'package:supabase_flutter/supabase_flutter.dart';

import '../models/note.dart';
import 'note_local_data_source.dart';
import 'note_remote_data_source.dart';

/// Hasil pengambilan daftar catatan.
class NotesResult {
  const NotesResult(this.notes, {required this.fromCache});

  final List<Note> notes;

  /// true jika server tidak bisa dihubungi dan data diambil dari Hive.
  final bool fromCache;
}

/// Satu pintu untuk semua data catatan. Controller tidak perlu tahu
/// apakah data berasal dari Supabase atau dari cache Hive.
class NoteRepository {
  NoteRepository({
    required this.remote,
    required this.local,
    this.timeout = const Duration(seconds: 10),
  });

  final NoteRemoteDataSource remote;
  final NoteLocalDataSource local;
  final Duration timeout;
```

```dart
  /// Ambil dari server lalu simpan ke cache. Jika gagal karena jaringan,
  /// kembalikan isi cache (mode offline).
  Future<NotesResult> getNotes() async {
    try {
      final notes = await remote.fetchNotes().timeout(timeout);
      await local.saveNotes(notes);
      return NotesResult(notes, fromCache: false);
    } on PostgrestException {
      // Server menjawab tapi menolak (misalnya RLS / tabel tidak ada).
      // Ini bukan masalah jaringan, jadi teruskan errornya.
      rethrow;
    } catch (_) {
      // SocketException, ClientException, TimeoutException, dll.
      return NotesResult(local.getNotes(), fromCache: true);
    }
  }

  Future<Note> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    final note = await remote
        .createNote(title: title, content: content, category: category)
        .timeout(timeout);
    await local.saveNote(note);
    return note;
  }

  Future<Note> updateNote(Note note) async {
    final updated = await remote.updateNote(note).timeout(timeout);
    await local.saveNote(updated);
    return updated;
  }

  Future<void> deleteNote(String id) async {
    await remote.deleteNote(id).timeout(timeout);
    await local.deleteNote(id);
  }

  Future<void> clearCache() => local.clear();
}
```

Controllers

lib/controllers/auth_controller.dart

```dart
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:get/get.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';

/// Hasil proses registrasi, dipakai RegisterView untuk menentukan
/// pesan yang ditampilkan.
enum RegisterResult { failed, loggedIn, needsEmailConfirmation }
```

```dart
class AuthController extends GetxController {
  AuthController(this._authService);

  final AuthService _authService;

  // State privat yang reaktif.
  final _isLoading = false.obs;
  final _errorMessage = RxnString();
  final _session = Rxn<Session>();

  StreamSubscription<AuthState>? _authSubscription;

  // Getter publik read-only. Membaca getter ini di dalam Obx
  // otomatis mendaftarkan Obx sebagai pendengar.
  bool get isLoading => _isLoading.value;
  String? get errorMessage => _errorMessage.value;

  Stream<AuthState> get authStateChanges => _authService.authStateChanges;
  Session? get currentSession => _session.value;
  User? get currentUser => _session.value?.user;

  /// Nama yang ditampilkan di AppBar. Diambil dari metadata saat register.
  String get displayName {
    final user = currentUser;
    if (user == null) return '';
    final name = user.userMetadata?['full_name'];
    if (name is String && name.isNotEmpty) return name;
    return user.email ?? '';
  }

  @override
  void onInit() {
    super.onInit();
    // Mulai dari sesi yang tersimpan di perangkat (jika ada) ...
    _session.value = _authService.currentSession;
    // ... lalu ikuti setiap perubahan status login.
    _authSubscription = _authService.authStateChanges.listen(
      (data) => _session.value = data.session,
      onError: (Object e) => debugPrint('authStateChanges: $e'),
    );
  }

  @override
  void onClose() {
    _authSubscription?.cancel();
    super.onClose();
  }

  Future<bool> signIn({required String email, required String password}) async {
    _setLoading(true);
    try {
      await _authService.signIn(email: email.trim(), password: password);
      return true;
    } on AuthException catch (e) {
      _errorMessage.value = _mapAuthError(e);
      return false;
    } catch (e) {
      _errorMessage.value = 'Terjadi kesalahan tak terduga: $e';
```

```
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<RegisterResult> register({
    required String fullName,
    required String email,
    required String password,
  }) async {
    _setLoading(true);
    try {
      final response = await _authService.signUp(
        fullName: fullName.trim(),
        email: email.trim(),
        password: password,
      );
      // Jika "Confirm email" aktif di dashboard, session bernilai null
      // sampai pengguna mengeklik link di email.
      if (response.session == null) {
        return RegisterResult.needsEmailConfirmation;
      }
      return RegisterResult.loggedIn;
    } on AuthException catch (e) {
      _errorMessage.value = _mapAuthError(e);
      return RegisterResult.failed;
    } catch (e) {
      _errorMessage.value = 'Terjadi kesalahan tak terduga: $e';
      return RegisterResult.failed;
    } finally {
      _setLoading(false);
    }
  }

  Future<void> signOut() async {
    try {
      await _authService.signOut();
    } on AuthException catch (e) {
      // signOut tetap menghapus sesi lokal walau request ke server gagal.
      debugPrint('signOut: ${e.message}');
    }
  }

  void clearError() {
    _errorMessage.value = null;
  }

  void _setLoading(bool value) {
    _isLoading.value = value;
    if (value) _errorMessage.value = null;
  }

  /// Menerjemahkan error Supabase Auth menjadi pesan berbahasa Indonesia.
  String _mapAuthError(AuthException e) {
    if (e is AuthRetryableFetchException) {
      return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
    }
    switch (e.code) {
```

```
      case 'invalid_credentials':
        return 'Email atau password salah.';
      case 'email_not_confirmed':
        return 'Email belum dikonfirmasi. Buka link verifikasi di inbox Anda.';
      case 'user_already_exists':
      case 'email_exists':
        return 'Email sudah terdaftar. Silakan login.';
      case 'weak_password':
        return 'Password terlalu lemah. Gunakan minimal 6 karakter.';
      case 'email_address_invalid':
        return 'Alamat email tidak valid.';
      case 'over_email_send_rate_limit':
      case 'over_request_rate_limit':
        return 'Terlalu banyak percobaan. Tunggu beberapa menit lalu coba lagi.';
      case 'signup_disabled':
        return 'Pendaftaran akun baru sedang dinonaktifkan.';
    }
    return e.message;
  }
}
```

lib/controllers/notes_controller.dart

```
import 'dart:async';

import 'package:get/get.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

class NotesController extends GetxController {
  NotesController(this._repository);

  final NoteRepository _repository;

  // State privat yang reaktif (.obs). Setiap perubahan otomatis
  // memberi tahu Obx yang membacanya.
  final _notes = <Note>[].obs;
  final _isLoading = false.obs;
  final _isOffline = false.obs;
  final _errorMessage = RxnString();
  final _query = ''.obs;
  final _categoryFilter = RxnString(); // null = semua kategori

  bool get isLoading => _isLoading.value;
  bool get isOffline => _isOffline.value;
  String? get errorMessage => _errorMessage.value;
  String get query => _query.value;
  String? get categoryFilter => _categoryFilter.value;
  bool get hasNotes => _notes.isNotEmpty;

  /// Daftar yang ditampilkan di layar setelah difilter pencarian & kategori.
  List<Note> get visibleNotes {
    final q = _query.value.toLowerCase();
    final kategori = _categoryFilter.value;
    return _notes.where((note) {
      final cocokKategori = kategori == null || note.category == kategori;
```

```
      final cocokCari =
          q.isEmpty ||
          note.title.toLowerCase().contains(q) ||
          note.content.toLowerCase().contains(q);
      return cocokKategori && cocokCari;
    }).toList();
  }

  Note? findById(String id) {
    for (final note in _notes) {
      if (note.id == id) return note;
    }
    return null;
  }

  /// Memuat catatan (dipanggil saat HomeView dibuka & saat pull-to-refresh).
  Future<void> loadNotes() async {
    _isLoading.value = true;
    _errorMessage.value = null;

    try {
      final result = await _repository.getNotes();
      _notes.assignAll(result.notes);
      _isOffline.value = result.fromCache;
    } catch (e) {
      _errorMessage.value = _friendlyError(e);
    } finally {
      _isLoading.value = false;
    }
  }

  /// Mengembalikan null jika berhasil, atau pesan error jika gagal.
  Future<String?> addNote({
    required String title,
    required String content,
    required String category,
  }) async {
    try {
      final note = await _repository.addNote(
        title: title,
        content: content,
        category: category,
      );
      _notes.insert(0, note);
      _isOffline.value = false;
      return null;
    } catch (e) {
      return _friendlyError(e);
    }
  }

  Future<String?> updateNote(Note note) async {
    try {
      final updated = await _repository.updateNote(note);
      _notes.removeWhere((n) => n.id == updated.id);
      _notes.insert(0, updated); // yang baru diubah naik ke atas
      _isOffline.value = false;
      return null;
    } catch (e) {
```

```
      return _friendlyError(e);
    }
  }

  /// Optimistic delete: hapus dari layar dulu, baru kirim ke server.
  /// Jika server gagal, catatan dikembalikan ke posisi semula (rollback).
  Future<String?> deleteNote(Note note) async {
    final index = _notes.indexWhere((n) => n.id == note.id);
    if (index == -1) return null;

    _notes.removeAt(index); // UI langsung diperbarui

    try {
      await _repository.deleteNote(note.id);
      return null;
    } catch (e) {
      _notes.insert(index, note); // rollback, UI ikut diperbarui
      return _friendlyError(e);
    }
  }

  void setQuery(String value) => _query.value = value;

  void setCategoryFilter(String? category) => _categoryFilter.value = category;

  /// Dipanggil saat logout: kosongkan state dan cache Hive.
  Future<void> clear() async {
    _notes.clear();
    _query.value = '';
    _categoryFilter.value = null;
    _errorMessage.value = null;
    _isOffline.value = false;
    await _repository.clearCache();
  }

  String _friendlyError(Object e) {
    if (e is PostgrestException) {
      if (e.code == '42501') {
        return 'Akses ditolak oleh Row Level Security. Periksa policy tabel notes.';
      }
      if (e.code == '42P01' || e.code == 'PGRST205') {
        return 'Tabel notes belum dibuat. Jalankan SQL di Supabase.';
      }
      return 'Kesalahan database: ${e.message}';
    }
    if (e is TimeoutException) {
      return 'Server terlalu lama merespons. Coba lagi.';
    }
    return 'Tidak dapat terhubung ke server. Periksa koneksi internet.';
  }
}
```

Views

lib/views/auth_gate.dart

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';
```

```dart
import '../controllers/auth_controller.dart';
import 'home_view.dart';
import 'login_view.dart';

/// "Penjaga pintu" aplikasi: mengamati status login di AuthController
/// lalu memilih halaman yang tepat secara otomatis.
class AuthGate extends GetView<AuthController> {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context) {
    return Obx(() {
      // currentSession dibaca di dalam Obx, jadi setiap kali sesi berubah
      // (signedIn, signedOut, ...) builder ini dijalankan ulang.
      if (controller.currentSession == null) {
        return const LoginView();
      }
      return const HomeView();
    });
  }
}
```

lib/views/login_view.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../utils/validators.dart';
import 'register_view.dart';

class LoginView extends StatefulWidget {
  const LoginView({super.key});

  @override
  State<LoginView> createState() => _LoginViewState();
}

class _LoginViewState extends State<LoginView> {
  final _auth = Get.find<AuthController>();
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    // Tutup keyboard lalu validasi form.
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;
```

```
    final success = await _auth.signIn(
      email: _emailController.text,
      password: _passwordController.text,
    );

    // Jika sukses, AuthGate otomatis pindah ke HomeView.
    if (!success && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(_auth.errorMessage ?? 'Login gagal'),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Center(
          child: SingleChildScrollView(
            padding: const EdgeInsets.all(24),
            child: Form(
              key: _formKey,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Icon(
                    Icons.menu_book_rounded,
                    size: 72,
                    color: Theme.of(context).colorScheme.primary,
                  ),
                  const SizedBox(height: 16),
                  Text(
                    'Catatan Kuliah',
                    textAlign: TextAlign.center,
                    style: Theme.of(context).textTheme.headlineMedium,
                  ),
                  const SizedBox(height: 8),
                  const Text(
                    'Masuk untuk melihat catatan Anda',
                    textAlign: TextAlign.center,
                  ),
                  const SizedBox(height: 32),
                  TextFormField(
                    controller: _emailController,
                    keyboardType: TextInputType.emailAddress,
                    textInputAction: TextInputAction.next,
                    autofillHints: const [AutofillHints.email],
                    decoration: const InputDecoration(
                      labelText: 'Email',
                      prefixIcon: Icon(Icons.email_outlined),
                      border: OutlineInputBorder(),
                    ),
                    validator: Validators.email,
                  ),
                  const SizedBox(height: 16),
                  TextFormField(
```

```
                controller: _passwordController,
                obscureText: _obscurePassword,
                textInputAction: TextInputAction.done,
                onFieldSubmitted: (_) => _submit(),
                decoration: InputDecoration(
                  labelText: 'Password',
                  prefixIcon: const Icon(Icons.lock_outline),
                  border: const OutlineInputBorder(),
                  suffixIcon: IconButton(
                    tooltip: _obscurePassword
                        ? 'Tampilkan password'
                        : 'Sembunyikan password',
                    icon: Icon(
                      _obscurePassword
                          ? Icons.visibility_outlined
                          : Icons.visibility_off_outlined,
                    ),
                    onPressed: () {
                      setState(() => _obscurePassword = !_obscurePassword);
                    },
                  ),
                ),
                validator: Validators.password,
              ),
              const SizedBox(height: 24),
              // Hanya bagian tombol yang bergantung pada isLoading,
              // jadi hanya bagian ini yang dibungkus Obx.
              Obx(() {
                final isLoading = _auth.isLoading;
                return Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  children: [
                    FilledButton(
                      onPressed: isLoading ? null : _submit,
                      style: FilledButton.styleFrom(
                        minimumSize: const Size.fromHeight(48),
                      ),
                      child: isLoading
                          ? const SizedBox(
                              width: 22,
                              height: 22,
                              child: CircularProgressIndicator(
                                strokeWidth: 2.5,
                              ),
                            )
                          : const Text('Masuk'),
                    ),
                    const SizedBox(height: 12),
                    TextButton(
                      onPressed: isLoading
                          ? null
                          : () => Get.to(() => const RegisterView()),
                      child: const Text('Belum punya akun? Daftar di sini'),
                    ),
                  ],
                );
              }),
            ],
          ),
```

```
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/views/register_view.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/auth_controller.dart';
import '../utils/validators.dart';

class RegisterView extends StatefulWidget {
  const RegisterView({super.key});

  @override
  State<RegisterView> createState() => _RegisterViewState();
}

class _RegisterViewState extends State<RegisterView> {
  final _auth = Get.find<AuthController>();
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _confirmController = TextEditingController();
  bool _obscurePassword = true;

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    _passwordController.dispose();
    _confirmController.dispose();
    super.dispose();
  }

  String? _validateConfirm(String? value) {
    if (value == null || value.isEmpty) return 'Ulangi password Anda';
    if (value != _passwordController.text) return 'Password tidak sama';
    return null;
  }

  Future<void> _submit() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    final result = await _auth.register(
      fullName: _nameController.text,
      email: _emailController.text,
      password: _passwordController.text,
    );
    if (!mounted) return;
```

```
    switch (result) {
      case RegisterResult.loggedIn:
        // Sesi langsung aktif. AuthGate di bawah halaman ini sudah berganti
        // ke HomeView, jadi cukup tutup halaman register.
        Get.back();
      case RegisterResult.needsEmailConfirmation:
        await showDialog<void>(
          context: context,
          builder: (dialogContext) => AlertDialog(
            title: const Text('Cek Email Anda'),
            content: Text(
              'Link konfirmasi sudah dikirim ke ${_emailController.text.trim()}. '
              'Klik link tersebut, lalu login dengan akun baru Anda.',
            ),
            actions: [
              FilledButton(
                onPressed: () => Navigator.of(dialogContext).pop(),
                child: const Text('Mengerti'),
              ),
            ],
          ),
        );
        if (mounted) Get.back();
      case RegisterResult.failed:
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(_auth.errorMessage ?? 'Registrasi gagal'),
            backgroundColor: Theme.of(context).colorScheme.error,
          ),
        );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Daftar Akun')),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                TextFormField(
                  controller: _nameController,
                  textCapitalization: TextCapitalization.words,
                  textInputAction: TextInputAction.next,
                  decoration: const InputDecoration(
                    labelText: 'Nama Lengkap',
                    prefixIcon: Icon(Icons.person_outline),
                    border: OutlineInputBorder(),
                  ),
                  validator: Validators.fullName,
                ),
                const SizedBox(height: 16),
                TextFormField(
                  controller: _emailController,
```

```
            keyboardType: TextInputType.emailAddress,
            textInputAction: TextInputAction.next,
            decoration: const InputDecoration(
              labelText: 'Email',
              prefixIcon: Icon(Icons.email_outlined),
              border: OutlineInputBorder(),
            ),
            validator: Validators.email,
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _passwordController,
            obscureText: _obscurePassword,
            textInputAction: TextInputAction.next,
            decoration: InputDecoration(
              labelText: 'Password',
              helperText: 'Minimal 6 karakter',
              prefixIcon: const Icon(Icons.lock_outline),
              border: const OutlineInputBorder(),
              suffixIcon: IconButton(
                icon: Icon(
                  _obscurePassword
                      ? Icons.visibility_outlined
                      : Icons.visibility_off_outlined,
                ),
                onPressed: () {
                  setState(() => _obscurePassword = !_obscurePassword);
                },
              ),
            ),
            validator: Validators.password,
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _confirmController,
            obscureText: _obscurePassword,
            textInputAction: TextInputAction.done,
            onFieldSubmitted: (_) => _submit(),
            decoration: const InputDecoration(
              labelText: 'Ulangi Password',
              prefixIcon: Icon(Icons.lock_reset_outlined),
              border: OutlineInputBorder(),
            ),
            validator: _validateConfirm,
          ),
          const SizedBox(height: 24),
          Obx(() {
            final isLoading = _auth.isLoading;
            return Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                FilledButton(
                  onPressed: isLoading ? null : _submit,
                  style: FilledButton.styleFrom(
                    minimumSize: const Size.fromHeight(48),
                  ),
                  child: isLoading
                      ? const SizedBox(
                          width: 22,
```

```
                              height: 22,
                              child: CircularProgressIndicator(
                                strokeWidth: 2.5,
                              ),
                            )
                          : const Text('Daftar'),
                    ),
                    const SizedBox(height: 12),
                    TextButton(
                      onPressed: isLoading ? null : () => Get.back(),
                      child: const Text('Sudah punya akun? Masuk'),
                    ),
                  ],
                );
              }),
            ],
          ),
        ),
      ),
    );
  }
}
```

lib/views/home_view.dart

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../models/note.dart';
import '../controllers/auth_controller.dart';
import '../controllers/notes_controller.dart';
import '../widgets/confirm_dialog.dart';
import '../widgets/note_card.dart';
import '../widgets/offline_banner.dart';
import 'note_detail_view.dart';
import 'note_form_view.dart';

class HomeView extends StatefulWidget {
  const HomeView({super.key});

  @override
  State<HomeView> createState() => _HomeViewState();
}

class _HomeViewState extends State<HomeView> {
  // Controller global yang didaftarkan di main() dengan Get.put.
  final _auth = Get.find<AuthController>();
  final _notes = Get.find<NotesController>();
  final _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
    // HomeView dibuat setiap kali user berhasil login, jadi inilah saat
    // yang tepat untuk memuat catatan milik user tersebut.
    _notes.loadNotes();
  }
```

```dart
@override
void dispose() {
  _searchController.dispose();
  super.dispose();
}

Future<void> _logout() async {
  final yakin = await showConfirmDialog(
    context,
    title: 'Keluar?',
    message: 'Catatan yang tersimpan di perangkat akan dihapus.',
    confirmLabel: 'Keluar',
  );
  if (!yakin) return;

  await _notes.clear(); // kosongkan state + cache Hive
  await _auth.signOut(); // AuthGate otomatis kembali ke LoginView
}

Future<bool> _confirmDelete(Note note) {
  return showConfirmDialog(
    context,
    title: 'Hapus catatan?',
    message: '"${note.title}" akan dihapus permanen.',
    confirmLabel: 'Hapus',
  );
}

Future<void> _delete(Note note) async {
  final messenger = ScaffoldMessenger.of(context);
  final error = await _notes.deleteNote(note);
  messenger.showSnackBar(
    SnackBar(
      content: Text(
        error == null
            ? 'Catatan dihapus'
            : 'Gagal menghapus, catatan dikembalikan. $error',
      ),
    ),
  );
}

void _openForm() => Get.to(() => const NoteFormView());

void _openDetail(Note note) => Get.to(() => NoteDetailView(noteId: note.id));

@override
Widget build(BuildContext context) {
  final name = _auth.displayName;

  return Scaffold(
    appBar: AppBar(
      title: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text('Catatan Kuliah'),
          Text('Halo, $name', style: Theme.of(context).textTheme.bodySmall),
        ],
```

```
        ),
        actions: [
          IconButton(
            tooltip: 'Keluar',
            icon: const Icon(Icons.logout),
            onPressed: _logout,
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _openForm,
        icon: const Icon(Icons.add),
        label: const Text('Catatan'),
      ),
      body: Column(
        children: [
          // Setiap Obx hanya rebuild jika state yang dibacanya berubah.
          Obx(
            () => _notes.isOffline
                ? OfflineBanner(onRetry: _notes.loadNotes)
                : const SizedBox.shrink(),
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 12, 16, 4),
            child: Obx(
              () => TextField(
                controller: _searchController,
                onChanged: _notes.setQuery,
                decoration: InputDecoration(
                  hintText: 'Cari judul atau isi catatan...',
                  prefixIcon: const Icon(Icons.search),
                  border: const OutlineInputBorder(),
                  isDense: true,
                  suffixIcon: _notes.query.isEmpty
                      ? null
                      : IconButton(
                          icon: const Icon(Icons.clear),
                          onPressed: () {
                            _searchController.clear();
                            _notes.setQuery('');
                          },
                        ),
                ),
              ),
            ),
          ),
          Obx(
            () => _CategoryFilter(
              selected: _notes.categoryFilter,
              onSelected: _notes.setCategoryFilter,
            ),
          ),
          Expanded(
            child: RefreshIndicator(
              onRefresh: _notes.loadNotes,
              child: Obx(_buildBody),
            ),
          ),
        ],
```

```
      ),
    );
  }

  Widget _buildBody() {
    // 1. Loading pertama kali (belum ada data sama sekali)
    if (_notes.isLoading && !_notes.hasNotes) {
      return const Center(child: CircularProgressIndicator());
    }

    // 2. Error dan tidak ada data cadangan
    if (_notes.errorMessage != null && !_notes.hasNotes) {
      return _MessageView(
        icon: Icons.error_outline,
        message: _notes.errorMessage!,
        action: FilledButton.icon(
          onPressed: _notes.loadNotes,
          icon: const Icon(Icons.refresh),
          label: const Text('Coba lagi'),
        ),
      );
    }

    final notes = _notes.visibleNotes;

    // 3. Kosong
    if (notes.isEmpty) {
      return _MessageView(
        icon: Icons.note_alt_outlined,
        message: _notes.hasNotes
            ? 'Tidak ada catatan yang cocok.'
            : 'Belum ada catatan.\nTekan tombol + untuk menambah.',
      );
    }

    // 4. Ada data
    return ListView.builder(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.only(top: 4, bottom: 88),
      itemCount: notes.length,
      itemBuilder: (context, index) {
        final note = notes[index];
        return Dismissible(
          key: ValueKey(note.id),
          direction: DismissDirection.endToStart,
          background: Container(
            alignment: Alignment.centerRight,
            padding: const EdgeInsets.only(right: 24),
            color: Theme.of(context).colorScheme.errorContainer,
            child: Icon(
              Icons.delete,
              color: Theme.of(context).colorScheme.onErrorContainer,
            ),
          ),
          confirmDismiss: (_) => _confirmDelete(note),
          onDismissed: (_) => _delete(note),
          child: NoteCard(note: note, onTap: () => _openDetail(note)),
        );
      },
```

```
    );
  }
}

/// Deretan chip untuk memfilter kategori.
class _CategoryFilter extends StatelessWidget {
  const _CategoryFilter({required this.selected, required this.onSelected});

  final String? selected;
  final ValueChanged<String?> onSelected;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 48,
      child: ListView(
        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 16),
        children: [
          Padding(
            padding: const EdgeInsets.only(right: 8),
            child: ChoiceChip(
              label: const Text('Semua'),
              selected: selected == null,
              onSelected: (_) => onSelected(null),
            ),
          ),
          for (final category in Note.categories)
            Padding(
              padding: const EdgeInsets.only(right: 8),
              child: ChoiceChip(
                label: Text(category),
                selected: selected == category,
                onSelected: (_) => onSelected(category),
              ),
            ),
        ],
      ),
    );
  }
}

/// Tampilan pesan di tengah layar yang tetap bisa ditarik (pull-to-refresh).
class _MessageView extends StatelessWidget {
  const _MessageView({required this.icon, required this.message, this.action});

  final IconData icon;
  final String message;
  final Widget? action;

  @override
  Widget build(BuildContext context) {
    return ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      padding: const EdgeInsets.all(32),
      children: [
        const SizedBox(height: 48),
        Icon(icon, size: 64, color: Theme.of(context).colorScheme.outline),
        const SizedBox(height: 16),
```

```
        Text(message, textAlign: TextAlign.center),
        if (action != null) ...[
          const SizedBox(height: 16),
          Center(child: action),
        ],
      ],
    );
  }
}
```

lib/views/note_form_view.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/notes_controller.dart';
import '../models/note.dart';

/// Satu halaman untuk dua keperluan:
/// - note == null  -> mode TAMBAH
/// - note != null  -> mode EDIT
class NoteFormView extends StatefulWidget {
  const NoteFormView({super.key, this.note});

  final Note? note;

  @override
  State<NoteFormView> createState() => _NoteFormViewState();
}

class _NoteFormViewState extends State<NoteFormView> {
  final _notes = Get.find<NotesController>();
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _titleController;
  late final TextEditingController _contentController;
  late String _category;
  bool _isSaving = false;

  bool get _isEdit => widget.note != null;

  @override
  void initState() {
    super.initState();
    // Mode edit: isi form dengan data lama. Mode tambah: form kosong.
    _titleController = TextEditingController(text: widget.note?.title ?? '');
    _contentController = TextEditingController(
      text: widget.note?.content ?? '',
    );
    _category = widget.note?.category ?? Note.categories.first;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }
```

```
  Future<void> _save() async {
    FocusScope.of(context).unfocus();
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
    final title = _titleController.text.trim();
    final content = _contentController.text.trim();

    final String? error;
    if (_isEdit) {
      error = await _notes.updateNote(
        widget.note!.copyWith(
          title: title,
          content: content,
          category: _category,
        ),
      );
    } else {
      error = await _notes.addNote(
        title: title,
        content: content,
        category: _category,
      );
    }

    if (!mounted) return;
    setState(() => _isSaving = false);

    final messenger = ScaffoldMessenger.of(context);
    if (error == null) {
      messenger.showSnackBar(
        SnackBar(
          content: Text(_isEdit ? 'Catatan diperbarui' : 'Catatan disimpan'),
        ),
      );
      Get.back(); // kembali ke halaman sebelumnya
    } else {
      messenger.showSnackBar(
        SnackBar(
          content: Text(error),
          backgroundColor: Theme.of(context).colorScheme.error,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(_isEdit ? 'Edit Catatan' : 'Catatan Baru')),
      body: SafeArea(
        child: Form(
          key: _formKey,
          child: ListView(
            padding: const EdgeInsets.all(16),
            children: [
              TextFormField(
                controller: _titleController,
                textInputAction: TextInputAction.next,
```

```
            maxLength: 100,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Judul wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 8),
          DropdownButtonFormField<String>(
            initialValue: _category,
            decoration: const InputDecoration(
              labelText: 'Kategori',
              border: OutlineInputBorder(),
            ),
            items: [
              for (final category in Note.categories)
                DropdownMenuItem(value: category, child: Text(category)),
            ],
            onChanged: (value) {
              if (value != null) setState(() => _category = value);
            },
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _contentController,
            minLines: 6,
            maxLines: 12,
            keyboardType: TextInputType.multiline,
            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              alignLabelWithHint: true,
              border: OutlineInputBorder(),
            ),
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return 'Isi catatan wajib diisi';
              }
              return null;
            },
          ),
          const SizedBox(height: 24),
          FilledButton.icon(
            onPressed: _isSaving ? null : _save,
            style: FilledButton.styleFrom(
              minimumSize: const Size.fromHeight(48),
            ),
            icon: _isSaving
                ? const SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2.5),
                  )
                : const Icon(Icons.save),
            label: Text(_isSaving ? 'Menyimpan...' : 'Simpan'),
```

```
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```

lib/views/note_detail_view.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/notes_controller.dart';
import '../models/note.dart';
import '../utils/date_format.dart';
import '../widgets/confirm_dialog.dart';
import 'note_form_view.dart';

class NoteDetailView extends GetView<NotesController> {
  const NoteDetailView({super.key, required this.noteId});

  final String noteId;

  Future<void> _delete(BuildContext context, Note note) async {
    final yakin = await showConfirmDialog(
      context,
      title: 'Hapus catatan?',
      message: '"${note.title}" akan dihapus permanen.',
      confirmLabel: 'Hapus',
    );
    if (!yakin || !context.mounted) return;

    final messenger = ScaffoldMessenger.of(context);
    // Tutup halaman detail dulu, baru hapus (optimistic delete).
    Get.back();
    final error = await controller.deleteNote(note);
    messenger.showSnackBar(SnackBar(content: Text(error ?? 'Catatan dihapus')));
  }

  @override
  Widget build(BuildContext context) {
    // Obx membaca versi terbaru catatan dari controller, sehingga setelah
    // diedit halaman ini langsung menampilkan perubahan.
    return Obx(() {
      final note = controller.findById(noteId);
      if (note == null) {
        return Scaffold(
          appBar: AppBar(),
          body: const Center(child: Text('Catatan tidak ditemukan.')),
        );
      }
      return _buildDetail(context, note);
    });
  }

  Widget _buildDetail(BuildContext context, Note note) {
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Detail Catatan'),
        actions: [
          IconButton(
            tooltip: 'Edit',
            icon: const Icon(Icons.edit),
            onPressed: () => Get.to(() => NoteFormView(note: note)),
          ),
          IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: () => _delete(context, note),
          ),
        ],
      ),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(note.title, style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              Chip(label: Text(note.category)),
              Text(
                'Diubah ${formatTanggal(note.updatedAt)}',
                style: Theme.of(context).textTheme.bodySmall,
              ),
            ],
          ),
          const Divider(height: 32),
          SelectableText(
            note.content,
            style: Theme.of(context).textTheme.bodyLarge,
          ),
        ],
      ),
    );
  }
}
```

Widgets

lib/widgets/confirm_dialog.dart

```
import 'package:flutter/material.dart';

/// Dialog konfirmasi Ya/Batal yang dipakai ulang di beberapa layar.
/// Mengembalikan true jika pengguna menekan tombol konfirmasi.
Future<bool> showConfirmDialog(
  BuildContext context, {
  required String title,
  required String message,
  required String confirmLabel,
}) async {
  final hasil = await showDialog<bool>(
```

```
    context: context,
    builder: (dialogContext) => AlertDialog(
      title: Text(title),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(dialogContext).pop(false),
          child: const Text('Batal'),
        ),
        FilledButton(
          onPressed: () => Navigator.of(dialogContext).pop(true),
          child: Text(confirmLabel),
        ),
      ],
    ),
  );
  return hasil ?? false;
}
```

lib/widgets/note_card.dart

```
import 'package:flutter/material.dart';

import '../models/note.dart';
import '../utils/date_format.dart';

class NoteCard extends StatelessWidget {
  const NoteCard({super.key, required this.note, this.onTap});

  final Note note;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      child: ListTile(
        onTap: onTap,
        contentPadding: const EdgeInsets.fromLTRB(16, 8, 16, 8),
        title: Text(
          note.title,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(fontWeight: FontWeight.w600),
        ),
        subtitle: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 4),
            Text(note.content, maxLines: 2, overflow: TextOverflow.ellipsis),
            const SizedBox(height: 8),
            Row(
              children: [
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 8,
```

```
                  vertical: 2,
                ),
                decoration: BoxDecoration(
                  color: scheme.primaryContainer,
                  borderRadius: BorderRadius.circular(12),
                ),
                child: Text(
                  note.category,
                  style: TextStyle(
                    fontSize: 12,
                    color: scheme.onPrimaryContainer,
                  ),
                ),
              ),
              const Spacer(),
              Text(
                formatTanggal(note.updatedAt),
                style: Theme.of(context).textTheme.bodySmall,
              ),
            ],
          ),
        ],
      ),
    ),
  );
  }
}
```

lib/widgets/offline_banner.dart

```
import 'package:flutter/material.dart';

/// Pita peringatan yang muncul ketika data diambil dari cache Hive.
class OfflineBanner extends StatelessWidget {
  const OfflineBanner({super.key, required this.onRetry});

  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;

    return Material(
      color: scheme.tertiaryContainer,
      child: Padding(
        padding: const EdgeInsets.fromLTRB(16, 4, 8, 4),
        child: Row(
          children: [
            Icon(Icons.cloud_off, color: scheme.onTertiaryContainer),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                'Mode offline: menampilkan data tersimpan.',
                style: TextStyle(color: scheme.onTertiaryContainer),
              ),
            ),
            TextButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
```

```
        ),
      ),
    );
  }
}
```

Utils

lib/utils/validators.dart

```dart
/// Kumpulan validator untuk TextFormField.
/// Setiap fungsi mengembalikan null jika valid, atau pesan error jika tidak.
class Validators {
  static String? fullName(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Nama lengkap wajib diisi';
    if (text.length < 3) return 'Nama minimal 3 karakter';
    return null;
  }

  static String? email(String? value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Email wajib diisi';
    final pola = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');
    if (!pola.hasMatch(text)) return 'Format email tidak valid';
    return null;
  }

  static String? password(String? value) {
    final text = value ?? '';
    if (text.isEmpty) return 'Password wajib diisi';
    if (text.length < 6) return 'Password minimal 6 karakter';
    return null;
  }
}
```

lib/utils/date_format.dart

```dart
/// Format tanggal sederhana tanpa package tambahan, contoh: 5 Okt 2026 14:07.
String formatTanggal(DateTime waktu) {
  const bulan = [
    'Jan',
    'Feb',
    'Mar',
    'Apr',
    'Mei',
    'Jun',
    'Jul',
    'Agu',
    'Sep',
    'Okt',
    'Nov',
    'Des',
  ];
  final lokal = waktu.toLocal();
  final jam = lokal.hour.toString().padLeft(2, '0');
  final menit = lokal.minute.toString().padLeft(2, '0');
```

```
  return '${lokal.day} ${bulan[lokal.month - 1]} ${lokal.year} $jam:$menit';
}
```

Test

test/helpers/fakes.dart

```dart
import 'package:catatan_kuliah/data/note_remote_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';

/// Membuat Note contoh untuk test. [hari] menentukan tanggal (urutan).
Note buatNote(
  String id, {
  String title = 'Judul',
  String category = 'Kuliah',
  int hari = 1,
}) {
  final waktu = DateTime.utc(2026, 10, hari);
  return Note(
    id: id,
    userId: 'user-1',
    title: title,
    content: 'Isi $title',
    category: category,
    createdAt: waktu,
    updatedAt: waktu,
  );
}

/// Pengganti Supabase untuk test. `implements` membuat kita wajib
/// menulis ulang semua method, sehingga tidak ada request jaringan sungguhan.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  List<Note> serverNotes = [];
  Object? errorToThrow; // jika diisi, setiap method melempar error ini

  void _maybeThrow() {
    final error = errorToThrow;
    if (error != null) throw error;
  }

  @override
  Future<List<Note>> fetchNotes() async {
    _maybeThrow();
    return serverNotes;
  }

  @override
  Future<Note> createNote({
    required String title,
    required String content,
    required String category,
  }) async {
    _maybeThrow();
    final note = buatNote('baru', title: title, category: category);
    serverNotes.add(note);
    return note;
  }
```

```
  @override
  Future<Note> updateNote(Note note) async {
    _maybeThrow();
    return note;
  }

  @override
  Future<void> deleteNote(String id) async {
    _maybeThrow();
    serverNotes.removeWhere((n) => n.id == id);
  }
}
```

test/note_local_data_source_test.dart

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;

  setUp(() async {
    // Hive.init (bukan initFlutter) karena test berjalan tanpa plugin Flutter.
    tempDir = await Directory.systemTemp.createTemp('hive_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('saveNotes lalu getNotes mengembalikan data urut terbaru', () async {
    await local.saveNotes([
      buatNote('a', title: 'Lama', hari: 1),
      buatNote('b', title: 'Baru', hari: 5),
    ]);

    final notes = local.getNotes();

    expect(notes.map((n) => n.id), ['b', 'a']);
    expect(notes.first.title, 'Baru');
  });

  test('saveNotes mengganti seluruh isi cache', () async {
    await local.saveNotes([buatNote('a'), buatNote('b')]);
    await local.saveNotes([buatNote('c')]);

    expect(local.getNotes().map((n) => n.id), ['c']);
  });
```

```
  test('saveNote, deleteNote, dan clear', () async {
    await local.saveNote(buatNote('a'));
    await local.saveNote(buatNote('a', title: 'Diubah'));
    expect(local.getNotes().single.title, 'Diubah');

    await local.saveNote(buatNote('b'));
    await local.deleteNote('a');
    expect(local.getNotes().map((n) => n.id), ['b']);

    await local.clear();
    expect(local.getNotes(), isEmpty);
  });
}
```

test/note_repository_test.dart

```
import 'dart:io';

import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late NoteLocalDataSource local;
  late FakeRemoteDataSource remote;
  late NoteRepository repository;

  setUp(() async {
    tempDir = await Directory.systemTemp.createTemp('repo_test_');
    Hive.init(tempDir.path);
    local = await NoteLocalDataSource.open();
    remote = FakeRemoteDataSource();
    repository = NoteRepository(remote: remote, local: local);
  });

  tearDown(() async {
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('online: data server dikembalikan dan disimpan ke cache', () async {
    remote.serverNotes = [buatNote('1'), buatNote('2')];

    final result = await repository.getNotes();

    expect(result.fromCache, isFalse);
    expect(result.notes, hasLength(2));
    expect(local.getNotes(), hasLength(2));
  });

  test('offline: jika jaringan gagal, data diambil dari cache', () async {
    await local.saveNotes([buatNote('cache-1', title: 'Dari Hive')]);
    remote.errorToThrow = const SocketException('Tidak ada internet');
```

```
    final result = await repository.getNotes();

    expect(result.fromCache, isTrue);
    expect(result.notes.single.title, 'Dari Hive');
  });

  test('error dari server (PostgrestException) tidak disembunyikan', () async {
    remote.errorToThrow = const PostgrestException(
      message: 'permission denied',
      code: '42501',
    );

    expect(repository.getNotes(), throwsA(isA<PostgrestException>()));
  });

  test('addNote dan deleteNote ikut memperbarui cache', () async {
    final note = await repository.addNote(
      title: 'Basis Data',
      content: 'Normalisasi',
      category: 'Kuliah',
    );
    expect(local.getNotes().single.id, note.id);

    await repository.deleteNote(note.id);
    expect(local.getNotes(), isEmpty);
  });

  test('deleteNote gagal: cache tidak berubah', () async {
    await local.saveNotes([buatNote('1')]);
    remote.errorToThrow = const SocketException('offline');

    await expectLater(
      repository.deleteNote('1'),
      throwsA(isA<SocketException>()),
    );
    expect(local.getNotes(), hasLength(1));
  });
}
```

test/notes_controller_test.dart

```
import 'dart:io';

import 'package:catatan_kuliah/controllers/notes_controller.dart';
import 'package:catatan_kuliah/data/note_local_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:get/get.dart';
import 'package:hive_ce/hive_ce.dart';

import 'helpers/fakes.dart';

void main() {
  late Directory tempDir;
  late FakeRemoteDataSource remote;
  late NotesController controller;
```

```
  setUp(() async {
    Get.testMode = true;
    tempDir = await Directory.systemTemp.createTemp('controller_test_');
    Hive.init(tempDir.path);
    remote = FakeRemoteDataSource()
      ..serverNotes = [
        buatNote('1', title: 'Basis Data', hari: 3),
        buatNote('2', title: 'Tugas Mobile', category: 'Tugas', hari: 2),
        buatNote('3', title: 'Kisi-kisi UTS', category: 'Ujian', hari: 1),
      ];
    final repository = NoteRepository(
      remote: remote,
      local: await NoteLocalDataSource.open(),
    );
    controller = Get.put(NotesController(repository));
  });

  tearDown(() async {
    Get.reset(); // hapus semua controller agar setiap test terisolasi
    await Hive.deleteFromDisk();
    await tempDir.delete(recursive: true);
  });

  test('loadNotes mengisi daftar dan mematikan loading', () async {
    final future = controller.loadNotes();
    expect(controller.isLoading, isTrue);

    await future;

    expect(controller.isLoading, isFalse);
    expect(controller.isOffline, isFalse);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('loadNotes offline: data cache dipakai dan isOffline true', () async {
    await controller.loadNotes(); // online dulu, cache terisi
    remote.errorToThrow = const SocketException('offline');

    await controller.loadNotes();

    expect(controller.isOffline, isTrue);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('setQuery dan setCategoryFilter menyaring visibleNotes', () async {
    await controller.loadNotes();

    controller.setQuery('mobile');
    expect(controller.visibleNotes.single.id, '2');

    controller.setQuery('');
    controller.setCategoryFilter('Ujian');
    expect(controller.visibleNotes.single.id, '3');

    controller.setCategoryFilter(null);
    expect(controller.visibleNotes, hasLength(3));
  });

  test('deleteNote gagal: catatan dikembalikan ke posisi semula', () async {
```

```
    await controller.loadNotes();
    final note = controller.findById('2')!;
    remote.errorToThrow = const SocketException('offline');

    final future = controller.deleteNote(note);
    // Optimistic: sebelum server menjawab, catatan sudah hilang.
    expect(controller.findById('2'), isNull);

    final error = await future;

    expect(error, isNotNull);
    expect(controller.visibleNotes.map((n) => n.id), ['1', '2', '3']);
  });
}
```

12. Troubleshooting & Ide Pengembangan

Daftar masalah yang paling sering ditemui mahasiswa saat mengerjakan proyek ini, beserta cara mendiagnosisnya. Biasakan membaca pesan error lengkap di konsol `flutter run` dan di **Logs** dashboard Supabase.

Tabel troubleshooting

Gejala / pesan	Penyebab	Solusi
`PostgrestException(code: 42501, message: new row violates row-level security policy for table "notes")`	Policy INSERT belum dibuat / salah, atau `user_id` tidak sama dengan `auth.uid()`	Jalankan ulang SQL section 1. Pastikan k `user_id` memiliki `default auth.uid` user sudah login
Daftar catatan selalu kosong padahal di Table Editor ada data	Policy SELECT tidak ada, atau data dibuat dengan `user_id` milik akun lain	Cek policy di Authentication → Policies, l bandingkan `user_id` baris dengan id d Authentication → Users
"Tabel notes belum dibuat" (code `PGRST205` / `42P01`)	SQL belum dijalankan atau nama tabel salah ketik	Jalankan SQL, lalu cek nama `notes` di `NoteRemoteDataSource._table`
You must initialize the supabase instance before calling Supabase.instance	`Supabase.instance.client` dipakai sebelum `await Supabase.initialize`	Periksa urutan di `main()` dan pastikan `await`
Layar "SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY belum diisi"	Run tanpa `--dart-define-from-file=env.json`	Tambahkan argumen run atau atur `laun`
Login gagal: *Email belum dikonfirmasi*	Confirm email aktif dan link belum diklik	Klik link di email, atau matikan Confirm e Authentication → Sign In / Providers → E
HiveError: Box not found. Did you forget to call Hive.openBox()?	Box dipakai sebelum dibuka	Pastikan `await NoteLocalDataSourc` dipanggil di `main()` setelah `Hive.initFlutter()`
"NotesController" not found. You need to call "Get.put(NotesController())"	`Get.find` dipanggil sebelum controller didaftarkan, atau `Get.put` lupa ditulis di `main()`	Pastikan kedua `Get.put(..., perman true)` ada di `main()` *sebelum* `runApp`
[Get] the improper use of a GetX has been detected	Ada `Obx` yang builder-nya tidak membaca satu pun nilai Rx (misalnya hanya membaca variabel biasa)	Pastikan di dalam `Obx` ada pemanggilan controller yang membaca `.value` /RxLis tidak butuh reaktif, hapus `Obx` -nya

Gejala / pesan	Penyebab	Solusi
Daftar tidak berubah setelah tambah/hapus padahal data di server berubah	List diganti dengan list biasa, atau objek di dalam RxList diubah langsung tanpa memberi tahu Obx	Ubah lewat method RxList (`assignAll` , `removeAt` , `clear`). Untuk mengganti s hapus lalu sisipkan versi barunya seperti `updateNote`
`Get.to` / `Get.back` tidak bekerja atau error *You are trying to use contextless navigation without a GetMaterialApp*	Root aplikasi masih `MaterialApp`	Ganti dengan `GetMaterialApp` di `CatatanKuliahApp`
A dismissed Dismissible widget is still part of the tree	Item tidak dihapus dari list setelah digeser	Pastikan `deleteNote` menghapus dari sebelum `await`
Banner offline tidak pernah muncul walaupun mode pesawat	Semua error ditangkap sebagai error biasa, bukan fallback	Periksa `NoteRepository.getNotes` : `PostgrestException` rethrow, `catch` kembalikan cache
Banner offline muncul padahal online (dan data lama)	Error database tersembunyi sebagai "offline", atau URL salah	Jalankan `flutter test` dan periksa lo URL di `env.json` benar
Setelah logout lalu login akun lain, catatan akun lama sempat terlihat	Cache/state tidak dibersihkan saat logout	Pastikan `_logout` memanggil `NotesController.clear()` *sebelum* `signOut()` . Ingat, `NotesController` `permanent` sehingga state-nya tidak hi
`updated_at` tidak berubah setelah edit	Trigger `notes_set_updated_at` belum dibuat	Jalankan bagian trigger di SQL
Insert gagal `23514` (check constraint)	Kategori di Dart berbeda dengan daftar di SQL, atau judul > 100 karakter	Samakan `Note.categories` dengan `c` `(category in (...))`
APK release tidak bisa konek, debug bisa	Izin INTERNET belum ada di manifest utama	Tambahkan `uses-permission INTERN` `android/app/src/main/AndroidMan`
Aplikasi tiba-tiba tidak bisa konek setelah lama tidak dipakai	Project Supabase gratis di-pause	Buka dashboard dan klik **Restore project**

Ide pengembangan (untuk tugas besar)

Ide	Petunjuk implementasi	Tingkat
Sinkronisasi realtime antar perangkat	Aktifkan Realtime untuk tabel `notes`, lalu di `NotesController` dengarkan `client.from('notes').stream(primaryKey: ['id'])` dan perbarui `_notes` (dengan `assignAll`) + cache setiap ada data baru. Simpan `StreamSubscription`-nya dan batalkan saat logout (di `clear()`) serta di `onClose()`.	Menengah
Antrean penulisan offline (offline queue)	Buat box Hive kedua `pending_ops` berisi `{type: 'insert'|'update'|'delete', data: {...}}`. Saat offline, simpan operasi ke antrean dan perbarui cache lokal. Saat online kembali (refresh sukses), kirim antrean secara berurutan. Gunakan id UUID buatan klien (package `uuid`) agar insert offline punya id sejak awal.	Sulit
Lampiran file (foto papan tulis, PDF)	Buat bucket `attachments` dengan policy folder per user (Modul 12 section 10), tambah kolom `attachment_path text` di `notes`, unggah memakai `uploadBinary`, dan tampilkan dengan `createSignedUrl`.	Menengah
Pin catatan penting	Tambah kolom `is_pinned boolean not null default false`, lalu urutkan `.order('is_pinned', ascending: false).order('updated_at', ascending: false)`.	Mudah
Relasi ke mata kuliah	Ganti `category` dengan `mata_kuliah_id bigint references mata_kuliah(id)` dan ambil nama dengan `select('*, mata_kuliah(nama)')`.	Menengah
Lupa password	`auth.resetPasswordForEmail(email)` + deep link, dengan event `passwordRecovery` di AuthGate.	Menengah
Halaman profil	Baca/ubah tabel `profiles` (`maybeSingle` + `update`), foto profil dari Storage.	Mudah
Pagination	`.range()` + muat lebih banyak saat scroll mendekati bawah.	Menengah

Ringkasan Modul 14

- Tabel `notes` dilindungi empat policy RLS berbasis `auth.uid()`, dengan `user_id default auth.uid()` dan trigger `updated_at`.
- Lapisan data: **Remote** (Supabase CRUD), **Local** (Hive Map dengan key id), dan **Repository** (network first, cache fallback, PostgrestException tidak disembunyikan).
- `NotesController` (GetX): state reaktif `.obs` untuk loading/error/offline, pencarian dan filter di memori, optimistic delete dengan rollback. Didaftarkan sekali di `main()` dengan `Get.put(...,`

`permanent: true)`.

- View: RefreshIndicator, empty/loading/error state, Dismissible + konfirmasi, satu form untuk tambah & edit, detail reaktif dengan `Obx` + `findById`, navigasi dengan `Get.to` / `Get.back`.
- Logout membersihkan state dan cache sebelum sesi dihapus.
- Logika offline dan `NotesController` diuji dengan unit test memakai fake data source, Hive di folder sementara, serta `Get.testMode` / `Get.reset`.

Tugas Praktikum: Praktikum 14: CRUD Catatan Kuliah dengan Supabase + Cache Offline

Sasaran Praktikum

Menyelesaikan aplikasi Catatan Kuliah dengan CRUD penuh ke Supabase, keamanan RLS per user, dan kemampuan membaca data saat offline dari cache Hive.

Langkah Pengerjaan

1. Jalankan SQL tabel `notes` + 4 policy RLS + trigger `updated_at`. Simpan di `supabase/setup.sql`.
2. Implementasikan `Note`, `NoteRemoteDataSource`, `NoteLocalDataSource`, dan `NoteRepository` (section 2–5).
3. Implementasikan `NotesController` (GetX) dengan pencarian, filter kategori, dan optimistic delete + rollback (section 6).
4. Ganti HomeView sementara dengan HomeView final, lalu buat NoteFormView (tambah & edit) dan NoteDetailView (section 7–8).
5. Perbarui `main.dart` (daftarkan `AuthController` dan `NotesController` dengan `Get.put`) dan pastikan logout membersihkan cache Hive (section 9).
6. Tambahkan file bantu dan tiga file unit test, lalu jalankan `flutter test` hingga semua lulus (section 10).
7. Uji RLS dengan dua akun berbeda: catatan akun A tidak boleh terlihat oleh akun B.
8. Uji mode offline: banner muncul, data cache tampil, tambah catatan gagal dengan pesan jelas, hapus catatan di-rollback.
9. Kerjakan minimal satu ide pengembangan dari section 12 (misalnya pin catatan atau realtime).

Output yang Diharapkan

Aplikasi lengkap: login/register, daftar catatan milik sendiri dengan pencarian dan filter, tambah/edit/hapus tersimpan di Supabase, banner mode offline dengan data dari Hive, logout membersihkan cache, flutter analyze bersih, dan flutter test lulus.

Rubrik Penilaian

SQL notes + RLS + trigger benar dan teruji 2 akun (20%), Lapisan data remote/local/repository + fallback offline (25%), NotesController: search, filter, optimistic delete + rollback (15%), View: list, refresh, empty/error/loading, Dismissible, form tambah/edit (20%), Unit test lulus (10%), Fitur pengembangan + laporan (10%).

Kuis Pemahaman

1. Mengapa kolom user_id pada tabel notes diberi 'default auth.uid()'?

A. Agar setiap catatan memiliki id unik

B. Agar database otomatis mengisi pemilik catatan dari token login, sehingga aplikasi tidak perlu mengirim user_id

C. Agar catatan bisa dibaca semua user

D. Agar RLS tidak diperlukan lagi

2. Dalam NoteRepository.getNotes(), apa yang terjadi jika remote melempar PostgrestException?

A. Data diambil dari cache Hive

B. Exception diteruskan (rethrow) karena itu error dari server, bukan masalah koneksi

C. Aplikasi otomatis logout

D. Exception diabaikan dan mengembalikan list kosong

3. Mengapa NoteLocalDataSource menyimpan catatan dengan key berupa id catatan?

A. Karena Hive hanya menerima key String

B. Agar update dan delete satu catatan bisa langsung dengan put(id)/delete(id) tanpa mencari indeks

C. Agar data terenkripsi

D. Agar urutan otomatis berdasarkan tanggal

4. Apa yang dimaksud dengan optimistic delete pada NotesController?

A. Menghapus data di server tanpa konfirmasi

B. Menghapus item dari UI lebih dulu, lalu mengirim ke server, dan mengembalikannya (rollback) jika server gagal

C. Menghapus seluruh catatan sekaligus

D. Menunda penghapusan sampai aplikasi ditutup

5. Mengapa loadNotes() dipanggil di initState HomeView, bukan di onInit() NotesController?

A. Karena onInit() tidak boleh berisi kode async

B. Karena NotesController didaftarkan di main() sebelum user login dan onInit() hanya berjalan sekali, sedangkan HomeView dibuat setiap kali user berhasil login

C. Karena Obx hanya bisa membaca data yang dimuat dari View

D. Agar data dimuat dua kali

6. Saat logout, mengapa NotesController.clear() dipanggil sebelum AuthController.signOut()?

A. Agar logout lebih cepat

B. Agar state dan cache Hive milik user lama sudah bersih sebelum AuthGate berpindah ke LoginView; NotesController bersifat permanent sehingga isinya tidak hilang sendiri

C. Karena signOut menghapus Hive secara otomatis

D. Urutannya tidak berpengaruh

7. Mengapa NoteDetailView menerima noteId lalu memanggil controller.findById(noteId) di dalam Obx, alih-alih menerima objek Note langsung?

A. Karena objek Note tidak bisa dikirim lewat Get.to

B. Agar halaman selalu menampilkan versi terbaru dari controller (misalnya setelah diedit); Obx hanya rebuild bila daftar catatan berubah, bukan saat query pencarian berubah

C. Agar memakai lebih sedikit memori

D. Karena GetView melarang parameter constructor

8. Di unit test, mengapa dipakai Hive.init(tempDir.path) dan bukan Hive.initFlutter()?

A. Karena Hive.init lebih cepat

B. Karena unit test berjalan tanpa plugin platform (path_provider), jadi folder penyimpanan harus ditentukan manual

C. Karena initFlutter sudah deprecated

D. Karena test tidak boleh menyimpan file

9. Di unit test NotesController, apa fungsi Get.reset() di tearDown?

A. Menghapus folder Hive sementara

B. Menghapus semua instance yang didaftarkan dengan Get.put sehingga setiap test mulai dari controller baru

C. Mengembalikan semua catatan ke server

D. Mematikan Get.testMode

Kunci Jawaban & Pembahasan

1. B — auth.uid() mengembalikan id user dari token request. Dengan default ini, insert cukup mengirim title/content/category, dan policy with check memastikan nilainya cocok.

2. B — PostgrestException berarti server menjawab tetapi menolak (misalnya RLS atau tabel tidak ada). Menyembunyikannya dengan data cache akan menutupi bug yang harus diperbaiki.

3. B — Key yang sama dengan primary key server membuat sinkronisasi cache sederhana: put menimpa versi lama dan delete menghapus tepat satu catatan.

4. B — UI diperbarui seolah-olah server pasti berhasil sehingga terasa instan. Jika gagal, item disisipkan kembali di index semula dan user diberi tahu.

5. B — Controller permanent dibuat sekali di awal aplikasi, saat belum ada sesi login. HomeView baru muncul setelah login (lewat AuthGate), jadi memuat di initState-nya menjamin data diambil untuk user yang sedang login, termasuk setelah berganti akun.

6. B — Begitu signOut selesai, AuthGate langsung membuang HomeView, tetapi NotesController tetap hidup. Membersihkan lebih dulu menjamin tidak ada catatan user lama yang tertinggal di memori maupun di disk.

7. B — Objek yang dikirim lewat constructor tidak ikut berubah setelah diedit. Dengan membaca findById di dalam Obx, detail reaktif terhadap RxList _notes saja, karena hanya Rx itu yang dibaca builder-nya.

8. B — initFlutter memakai path_provider untuk mencari folder aplikasi di HP. Di lingkungan test, kita memberi folder sementara secara eksplisit lalu menghapusnya di tearDown.

9. B — Get.put menyimpan instance di penyimpanan global GetX. Tanpa Get.reset, controller dari test sebelumnya (beserta state-nya) bisa terbawa ke test berikutnya.